

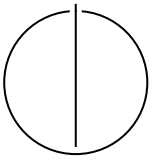
SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

A Case Study of Live Migration for Cloud Databases: Performance and Security Implications

Maximilian Eberl





SCHOOL OF COMPUTATION,
INFORMATION AND TECHNOLOGY —
INFORMATICS

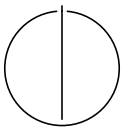
TECHNISCHE UNIVERSITÄT MÜNCHEN

Bachelor's Thesis in Informatics

A Case Study of Live Migration for Cloud Databases: Performance and Security Implications

Eine Fallstudie zur Live-Migration für Cloud-Datenbanken: Auswirkungen auf Leistung und Sicherheit

Author:	Maximilian Eberl
Examiner:	Prof. Dr. Viktor Leis
Supervisor:	M.Sc. Bohyun Lee
Submission Date:	23 July 2026



Declaration of Authorship

I hereby declare that I have written this thesis independently and have not used any sources or aids other than those specified. All passages taken directly or indirectly from published or unpublished sources have been appropriately acknowledged.

This thesis has not previously been submitted elsewhere for examination purposes.

Starnberg, 23 July 2026



Maximilian Eberl

Abstract

For modern cloud and serverless database providers, there is often a need to relocate active compute nodes. They can either preserve the valuable virtual machine (VM) running state via live migration or reconstruct the service via restarts or database-level mechanisms. It is often unclear whether the costs and risks of live migration are justified or whether a simpler alternative is preferable, as each method offers distinct advantages and drawbacks.

This thesis compares QEMU/KVM precopy, postcopy, and stop-copy migration with cold restart, restart with PostgreSQL cache prewarming, and a prepared PostgreSQL destination. The mechanisms are evaluated using moderate-load and stress-load variants of an update-only YCSB workload on a PostgreSQL VM, considering interruption, performance, resource overhead, and security and failure implications.

Precopy caused little disruption when it converged, but became inefficient when the dirty-page rate approached the available transfer capacity. Postcopy bounded migration duration under stress, but introduced demand-paging overhead after the switchover and stronger failure and security dependencies. Restart-based approaches were simpler but caused longer interruptions, and cache prewarming did not improve immediate throughput. A prepared destination shortened the interruption relative to restart but imposed substantial preparation overhead and storage contention. The results support precopy as the initial state-preserving mechanism, with postcopy as a fallback when convergence becomes unlikely and convergence-control mechanisms as alternatives when temporary workload interference or additional resource consumption is acceptable. Reconstruction-based approaches remain attractive when operational simplicity or reduced migration-specific exposure matters more than preserving complete VM state.

Acknowledgments

I would like to sincerely thank my supervisor, Bohyun Lee, for giving me the opportunity to explore a topic that I found genuinely fascinating. I especially enjoyed our meetings, where we discussed my findings and often found another question or angle worth exploring. Her genuine interest in the results made the research process much more enjoyable and helped shape the thesis's direction. I am very grateful for her guidance and honest feedback throughout the project.

I am also grateful to Prof. Dr. Viktor Leis for the opportunity to conduct this thesis at the Chair for Decentralized Information Systems and Data Management.

Finally, my thanks go to my friends for reading parts of the thesis, providing valuable feedback, and keeping me motivated, especially during the final stages.

Contents

Abstract iii

Acknowledgments iv

List of Figures viii

List of Tables ix

Abbreviations x

1 Introduction 1

2 Background 3
2.1 Serverless Database Compute and Relocation . . . 3
2.2 Virtualization with QEMU/KVM . . . 3
2.2.1 The QEMU/KVM Architecture . . . 4
2.2.2 Virtual Machine State . . . 4
2.2.3 The QEMU Machine Protocol . . . 4
2.3 Virtual Machine Live Migration . . . 6
2.3.1 Migration Model and Common Phases . . . 6
2.3.2 Precopy Migration . . . 7
2.3.3 Postcopy Migration . . . 7
2.3.4 Full Stop-Copy Migration . . . 8
2.3.5 Convergence Control and Hybrid Strategies . . . 9
2.4 Alternatives to Live Migration . . . 9
2.4.1 Plain Restart . . . 10
2.4.2 Restart with Cache Prewarming . . . 10
2.4.3 Prepared Destination Switchover . . . 11
2.5 PostgreSQL Memory and I/O Characteristics . . . 11
2.6 Security and Failure Considerations . . . 12
2.6.1 Failure Dependencies . . . 12
2.6.2 Migration State and Trust Boundaries . . . 13
2.6.3 Postcopy Migration and `Userfaultfd` . . . 13

2.6.4 Consistency and Split-Brain Risks . . . 14

3 Related Work **15**
3.1 Virtual-Machine Live Migration . . . 15
3.2 Database-Level Relocation . . . 15
3.3 Security and Failure Considerations . . . 16
3.4 Positioning of this thesis . . . 17

4 Methodology **18**
4.1 Research Design . . . 18
4.2 Experimental Environment . . . 18
4.3 Relocation Strategies . . . 20
4.3.1 VM Migration Strategies . . . 20
4.3.2 Restart-Based Strategies . . . 21
4.3.3 Prepared PostgreSQL Destination . . . 22
4.4 Experimental Procedure and Measurement . . . 22
4.5 Data Processing . . . 25

5 Evaluation **26**
5.1 Baseline . . . 26
5.2 Migration under Moderate Load . . . 28
5.2.1 Precopy Migration . . . 28
5.2.2 Immediate Postcopy Migration . . . 28
5.2.3 Stop-Copy Migration . . . 29
5.3 Migration under Stress . . . 31
5.3.1 Precopy Near the Convergence Limit . . . 31
5.3.2 Postcopy under Stress . . . 32
5.4 Convergence-Control Strategies . . . 34
5.4.1 Delayed Postcopy . . . 34
5.4.2 Automatic Convergence . . . 35
5.5 Restart Alternatives . . . 37
5.5.1 Cold and Prewarmed Restart . . . 37
5.5.2 Prepared PostgreSQL Destination . . . 39
5.6 Evaluation Summary . . . 41

6 Discussion and Conclusion **42**
6.1 State Preservation versus Reconstruction . . . 42
6.2 Service Interruption . . . 43
6.3 Resource Overheads . . . 43
6.4 Failure and Security Exposure . . . 44

6.5 Final Comparison 44
6.6 Answer to the Research Question 45
6.7 Limitations and Future Work 46

Bibliography **47**

Declaration of Generative AI Use **51**

List of Figures

2.1 Simplified architecture of the QEMU/KVM virtualization stack. 5

5.1 Throughput of the baseline workloads. 27
5.2 Throughput of the main migration strategies under moderate load . . . 30
5.3 Throughput of the main migration strategies under stress load 33
5.4 Throughput of the hybrid migration strategies under stress load 36
5.5 Throughput and buffer hit-rate comparison of cold vs. prewarmed restart. 38
5.6 Throughput and I/O pressure of the prepared destination scenario . . . 40

List of Tables

4.1 Host and VM configuration 19
4.2 Database and Workload configuration 19
4.3 Relevant QMP commands used to manage the guest systems 20
4.4 Relevant experimental metrics 24
4.5 Experimental setup 24

5.1 Migration results under moderate load. Values are mean ± standard deviation unless reported as median [IQR]. 28
5.2 Migration results under the sixteen-thread stress workload. Values are mean ± standard deviation unless reported as median [IQR]. 31
5.3 Delayed-postcopy results under the sixteen-thread workload. Values are mean ± standard deviation unless reported as median [IQR]. 34
5.4 Cold and prewarmed restart results. Values are mean ± standard deviation. 37

6.1 Qualitative comparison of the evaluated basic relocation mechanisms . 45

Abbreviations

VM Virtual Machine
KVM Kernel-based Virtual Machine
QMP QEMU Machine Protocol
YCSB Yahoo! Cloud Serving Benchmark
vCPU Virtual Central Processing Unit
WAL Write-Ahead Log
THP Transparent Huge Page
RSS Resident Set Size
IQR Interquartile Range

1 Introduction

Serverless database systems aim to make compute elastic and operationally flexible. Rather than assigning each workload permanently to a fixed physical machine, a provider may operate isolated compute instances across a shared fleet of hosts. Maintenance, load balancing, hardware changes, failures, or changes in resource demand can then require that an active database compute node be replaced, restarted, or moved. This is especially relevant because modern cloud database architectures often separate persistent storage from compute, allowing compute instances to be recreated or coordinated independently of the storage layer [Ver+17; Ant+19]. However, compute nodes are not entirely disposable. Guest memory, operating system state, PostgreSQL buffers, active processes, and connection-related state can continue to affect service behavior after persistent database state is available elsewhere.

Relocation mechanisms differ primarily in how they handle this volatile runtime state. A restart discards it and requires the destination to reconstruct the state needed by the workload. Application-level mechanisms may prepare a destination or restore selected state before handover. Live migration instead attempts to preserve the execution state of the complete virtual machine. This can minimize interruption and avoid reconstruction, but requires memory transfer, coordination, and protection of the transferred state. For an update-intensive database workload, relocation quality cannot be reduced to the period of complete unavailability. Different mechanisms may impose costs while the service remains active, during the transition, or after execution resumes. A meaningful comparison must therefore consider service disruption together with performance, resource, operational, failure, and security implications.

From the perspective of a serverless database provider, live migration is attractive but not necessarily preferable. It can preserve a warm compute instance and minimize interruption, but also introduces memory-transfer costs, bandwidth demands, migration-specific failure dependencies, and additional security requirements. Restarting compute or switching to a prepared database-level destination may be operationally simpler, even if these alternatives preserve less runtime state.

This raises the central question of this thesis: How does live migration using the QEMU/KVM virtualization stack compare with alternative database relocation mechanisms, and under which operational conditions is it justified to preserve the complete VM runtime state?

This study provides a mechanism-level evaluation motivated by this provider use case rather than an end-to-end evaluation of a production serverless database architecture. It compares QEMU/KVM precopy and postcopy migration, a full stop-copy baseline, a plain VM restart, a restart with PostgreSQL cache prewarming, and a prepared destination maintained via PostgreSQL streaming replication. Selected convergence-control variants are included to examine poorly converging migration. The experiments use an update-only YCSB workload on a PostgreSQL virtual machine and evaluate the mechanisms in terms of service disruption, workload performance, resource costs, and operational, failure, and security implications.

This thesis makes three principal contributions. First, it provides a controlled comparison of state-preserving, restart-based, and database-level relocation mechanisms under a common experimental setup. Second, it characterizes the workload-dependent convergence and performance behavior of precopy, postcopy, delayed postcopy, and automatic convergence. Third, it combines the experimental findings with a qualitative assessment of operational complexity, failure dependencies, and security exposure to support the provider-level decision between preserving and reconstructing runtime state.

Chapter 2 introduces the architectural and technical background required for the study. Chapter 3 reviews existing research and positions the thesis, while Chapter 4 describes the experimental methodology. Chapter 5 presents the results, and Chapter 6 interprets the resulting trade-offs, answers the research question, and discusses limitations and future work.

2 Background

The evaluation in this thesis involves mechanisms that operate at several layers of the system. At the architectural level, serverless and cloud-native database systems increasingly separate persistent database state from the compute nodes that execute queries and transactions. At the virtualization level, QEMU and KVM provide the virtual machine environment, while QEMU's migration mechanisms determine how running VM state is transferred from source to destination. At the database level, PostgreSQL's use of memory and storage influences the cost of relocation and the performance observed during and after the transition. This chapter introduces these components, along with alternatives to live migration and the security and failure properties considered later in the thesis.

2.1 Serverless Database Compute and Relocation

The separation of persistent database state from compute nodes allows running instances to be started, stopped, scaled, replaced, or moved while durable database state remains available through a separate storage or replication layer. Systems such as Aurora, Socrates, Neon, and HorizonDB illustrate this broader architectural direction [Ver+17; Ant+19; Neo; Mic]. This architecture makes live migration one possible implementation choice rather than an inherent requirement. A provider can recreate or prepare compute elsewhere, or attempt to preserve the running compute instance through virtual machine migration.

2.2 Virtualization with QEMU/KVM

System virtualization abstracts the physical resources of a host machine and exposes them to one or more isolated virtual machines (VMs). Each VM is presented with virtualized computing resources, such as processors, memory, storage, network interfaces, and other I/O devices, on which a guest operating system can execute mostly independent of the host system and other guests. A virtual machine monitor (VMM), commonly referred to as a hypervisor, manages this abstraction and mediates access to the underlying physical hardware.

2.2.1 The QEMU/KVM Architecture

The experiments in this thesis use the QEMU/KVM virtualization stack on Linux. QEMU is a machine emulator and virtual machine monitor that provides the virtual hardware environment in which a guest operating system runs. It manages the virtual machine process, guest memory, and emulated or virtualized devices such as storage and network interfaces. Although QEMU can emulate guest processors entirely in software, it can also use KVM for hardware-assisted virtualization.

KVM, short for Kernel-based Virtual Machine, is a virtualization component of the Linux kernel. It exposes the processor's hardware-virtualization facilities to userspace and allows guest virtual CPUs to execute directly on the physical processor under kernel control. In the combined QEMU/KVM stack, QEMU provides virtual-machine management and device emulation, while KVM accelerates guest CPU execution [QEMh; Linb].

2.2.2 Virtual Machine State

This division of responsibilities is directly relevant to live migration. A running VM cannot be reduced to the contents of its virtual disk. Continuing execution on another host also requires its runtime state, including its hardware configuration, guest memory, virtual CPU state, and virtual device state, to be reproduced at the destination. Persistent storage is a separate concern and may be shared, replicated, or transferred independently. In the QEMU/KVM stack considered in this thesis, QEMU coordinates the transfer of runtime state, while KVM continues to provide hardware-assisted guest execution on the source and destination hosts [QEMc].

2.2.3 The QEMU Machine Protocol

QEMU exposes the QEMU Machine Protocol (QMP) for programmatic management of running virtual machines. It uses structured JSON-based messages for commands, responses, and asynchronous events and can be accessed through interfaces such as Unix domain sockets [QEMf]. The experimental framework developed for this thesis uses QMP to directly control the source and destination QEMU processes. Details of the experimental implementation are provided in Chapter 4.

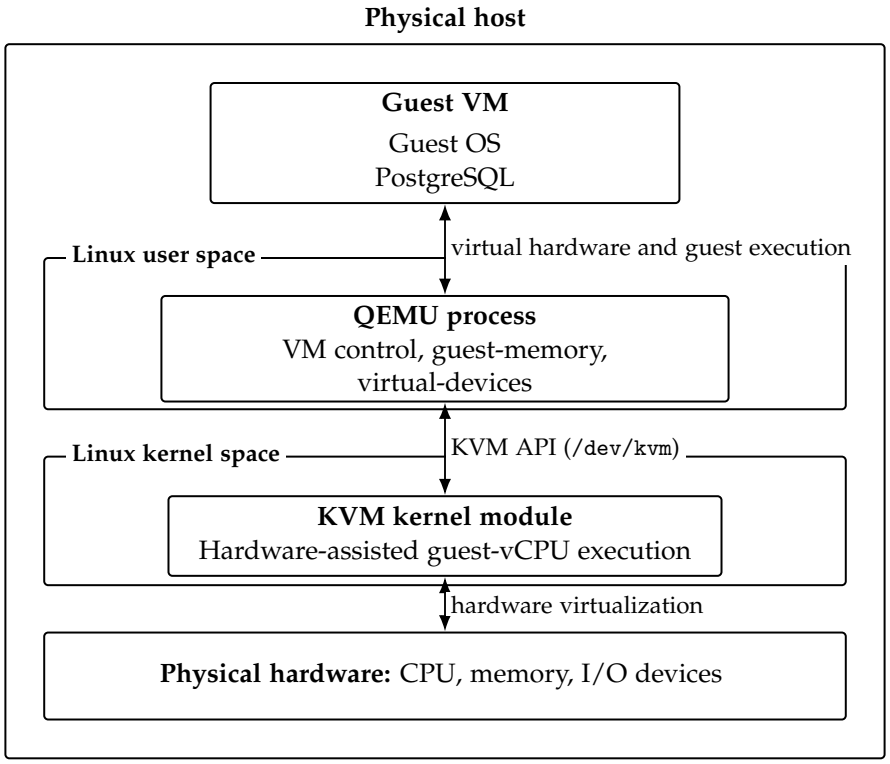


Figure 2.1: Simplified architecture of the QEMU/KVM virtualization stack.

2.3 Virtual Machine Live Migration

Virtual machine live migration describes the transfer of a running virtual machine from a source to a destination while preserving its execution state and minimizing service interruption. In contrast to restarting a VM on the same or another host, live migration aims to continue execution from the state reached at the source rather than from a newly initialized system. The central challenge of live migration is that the state to be transferred may continue to change while the VM is running. A migration mechanism must therefore determine which parts of the VM state are transferred, how they are transferred, and when execution moves to the destination. Different migration strategies make different trade-offs between total migration duration, service downtime, resource overhead, and failure exposure.

2.3.1 Migration Model and Common Phases

A migration involves a source instance, on which the VM initially executes, and a destination instance that can resume the transferred state. The source and destination must provide a sufficiently compatible execution environment for the migrated VM state to remain valid [QEMc].

For the purpose of this thesis, the migration is divided into several conceptual events. The *migration start* marks the point at which the transfer is initiated. The *switchover* occurs when execution stops at the source and begins at the destination. The interval during which the workload cannot make progress because of this transition is referred to as *downtime*. The *migration completion* marks the point at which the migration mechanism no longer depends on further state transfer between source and destination. These events do not necessarily occur within a short time frame of each other. In precopy migration, most memory transfer occurs before the switchover, whereas in postcopy migration, a substantial portion of the transfer may continue after execution has resumed at the destination [Cla+05; HG09].

Migration preserves the contents and guest-visible organization of memory, but not necessarily every property of its host-side representation. The destination may, for example, back the same guest memory using a different page-size composition. This distinction becomes relevant when interpreting the postcopy results in Chapter 5.2.2.

Persistent storage is treated independently of the runtime state transfer in the experiments presented in this thesis. The evaluated migration mechanisms primarily transfer volatile VM state, while the required persistent storage is provided by the storage configuration for the corresponding scenario. Migrating memory and persistent disk contents introduces different mechanisms and performance costs. Persistent storage migration is out of the scope of this thesis.

2.3.2 Precopy Migration

Precopy migration transfers guest memory while the source VM remains active. In an initial transfer phase, all memory pages are marked as dirty, and the hypervisor starts copying dirty pages from the source to the destination without stopping guest execution. Pages modified by the guest during this transfer become dirty again and must be transferred repeatedly in subsequent iterations. The process continues until the migration mechanism decides that the remaining state can be transferred during a final interruption of guest execution. During this final phase, the source VM is stopped, and its remaining memory and execution state are transferred to the destination VM. Once the required state is available, the VM resumes execution at the destination. The duration of this final phase determines the perceived service interruption for the workload [Cla+05].

The effectiveness of precopy migration depends on the relationship between the available transfer capacity and the rate at which the guest modifies memory. If dirty pages are produced slower than they can be transferred, the remaining amount of state decreases between migration rounds, and the migration can converge towards a relatively short final transfer. A write-intensive workload can instead cause pages to be modified repeatedly before migration completes. This increases the total amount of transferred data and may considerably extend the migration duration.

Precopy allows the source VM to continue executing for most of the migration, helping keep downtime low when the workload permits efficient convergence. Its main disadvantage is the repeated retransmission of memory pages that are modified during migration, which can drastically increase both the transfer volume and the total migration duration. If the rate at which the guest dirties memory approaches or exceeds the effective transfer rate, the migration may fail to converge. The migration mechanism must then either reduce the guest's rate of memory modification, for example by throttling virtual CPU execution, or force the switchover with a larger amount of remaining state, thereby increasing downtime. Mechanisms for managing this convergence problem are discussed in Section 2.3.5 [Ako+10].

2.3.3 Postcopy Migration

Postcopy migration changes the order of execution and memory transfer. Instead of transferring most of the guest memory before the switchover, postcopy first transfers only the minimum state required to resume execution at the destination. The VM then continues running at the destination while memory that has not yet been transferred remains available at the source.

If the guest accesses a page that is not yet present at the destination, execution cannot

continue until the page's contents have been retrieved from the source. The missing page is transferred on demand, and execution is blocked, while the remaining memory is still being transferred in the background. As a result, guest execution can move to the destination before the complete memory image has arrived. QEMU applies separate bandwidth parameters to the precopy and postcopy phases. Consequently, enabling postcopy does not necessarily imply that both phases operate under the same transfer-rate limit [QEMg].

This approach avoids repeated transmission of pages that would otherwise be modified during iterative precopy rounds. It can therefore provide a more predictable point of switchover for workloads that prevent precopy from converging quickly. The cost is shifted to the period after the switchover, where accesses to unavailable pages may stall guest execution until the required state arrives.

Postcopy can also change the host-side representation of guest memory. QEMU's postcopy state machine passes through an *advise–discard–listen–running* sequence. Before processing the first discard command, the destination uses `madvise()` to disable the creation of new huge pages during postcopy and to break up huge pages that overlap discarded memory ranges. The remaining memory is subsequently populated at page granularity through background transfer or `userfaultfd`-assisted retrieval. After migration, Linux may gradually collapse eligible base-page regions into transparent huge pages again. The guest-visible memory contents remain unchanged, but the temporary change in host page size composition may increase address-translation overhead [QEMe; Linc].

More generally, postcopy changes the migration's failure properties. After the switchover, the VM executes at the destination while parts of its memory state may still exist only at the source. Successful completion, therefore, depends on continued access to both sides until the remaining state has been transferred. In the QEMU implementation used in this thesis, fetching missing memory at the destination involves userspace-assisted page-fault handling through `userfaultfd`. The operational and security implications of these dependencies are further discussed in Section 2.6.

2.3.4 Full Stop-Copy Migration

The stop-copy baseline suspends the source VM before the main state transfer begins. The complete runtime state is then transferred while execution remains paused, and the VM resumes only after the required state has arrived at the destination. This thesis uses the term *stop-copy baseline* to distinguish this scenario from the final stop-and-copy phase of precopy migration. It is essentially a pure offline migration, also known as a cold migration. This approach is used to show an intentionally extreme baseline that eliminates the convergence problem by suspending the VM before transfer.

In standard precopy, the final interruption occurs after one or more transfer rounds, during which the VM continues to run. As the stop-copy baseline omits these rounds, the guest cannot modify memory while suspended, and pages do not need to be retransmitted during execution, removing the convergence problem. The cost is that most of the state transfer contributes directly to service downtime.

2.3.5 Convergence Control and Hybrid Strategies

The preceding migration strategies place the switchover at different points in the state transfer. In practice, migration systems also provide mechanisms that alter this trade-off without introducing a completely separate migration strategy.

Several mechanisms can improve the convergence of precopy migration without abandoning the approach entirely. QEMU's *auto-converge* mechanism progressively throttles virtual CPU execution when migration fails to converge, reducing the rate at which the guest can dirty memory at the cost of guest performance. *Dirty-rate limiting* pursues a similar goal more selectively by restricting the dirtying rate of individual virtual CPUs. Increasing the *migration bandwidth* instead aims to transfer state more quickly at the cost of potentially greater contention for host resources. These mechanisms trade workload performance and host resource usage against migration progress [QEMg; QEMa].

A different compromise is to begin migration using precopy and switch to postcopy if the iterative transfer fails to converge within an acceptable period. This retains the opportunity for precopy to complete with low downtime under favorable workload conditions while providing a way to bound the time spent retransmitting modified pages. Once the migration switches to postcopy, however, it also inherits the failure dependencies and page-fault-related performance costs of that approach [Svä+15; QEMe].

2.4 Alternatives to Live Migration

Preserving the complete runtime state is not the only way to relocate a database workload. A service can instead be stopped and recreated on another VM, or a destination can be prepared before the transition, so that only the remaining application state needs to be synchronized. Such approaches avoid the need to transfer the complete volatile state of a running VM but introduce costs associated with initialization, cache warmup, state synchronization, or even greater host overhead.

2.4.1 Plain Restart

The simplest baseline alternative to live migration is to stop the source VM and start a new instance at the destination. This approach requires no migration protocol, no transfer of guest memory, and little migration-specific configuration. Apart from making the required persistent data available, the destination can be started using the same mechanisms as any newly created VM.

The destination has to initialize the operating system and rebuild performance-relevant state such as database and operating system caches. For workloads that depend on preserving in-memory state or uninterrupted execution, this trade-off may be unacceptable regardless of the transition time. However, avoiding the transfer of volatile VM state can also be advantageous when the migration path is bandwidth-constrained. Live migration may require several gigabytes of memory to be transferred and, in the case of precopy migration, modified pages may have to be transmitted repeatedly. In the case of postcopy migration, latency can substantially impair destination execution when on-demand pages stall the destination for too long during transfer. Under such conditions, migration can take considerably longer than starting a new VM at the destination and may impose greater performance costs than a cold boot on a new instance. A restart may therefore provide a shorter transition even though the destination subsequently has to rebuild performance- or execution-relevant state in memory.

2.4.2 Restart with Cache Prewarming

Although the database may become available shortly after startup, its performance can remain below that of the source instance while frequently accessed data is loaded back into memory. Cache prewarming attempts to reduce this effect by explicitly reconstructing selected performance-relevant buffers before or shortly after the service resumes.

The implementation of cache prewarming depends more on the database system and workload than on hypervisor features. PostgreSQL provides the `pg_prewarm` extension for loading relation data into the operating-system cache or PostgreSQL's shared buffers. When automatic prewarming is enabled, a background worker periodically records which relation blocks are currently resident in the shared buffers in a file named `autoprewarm.blocks`. After a restart, PostgreSQL uses this record to load the corresponding blocks back into its shared buffers [Posc]. The file does not contain the cached data itself, but rather identifies the blocks that should be read again from persistent storage. In effect, PostgreSQL retains a description of its previous buffer contents and uses it to reconstruct part of the warm database state after startup. This

introduces storage I/O and consumes time after PostgreSQL starts on the destination again.

2.4.3 Prepared Destination Switchover

A more extensive alternative is to prepare a destination instance before the source stops serving the workload. The destination VM can complete its boot process and initialize the database in advance, moving these costs outside of the final transition. Unlike live migration, the destination does not continue from the exact execution state of the source VM. Instead, it maintains a separate runtime environment that is brought sufficiently close to the source's application state to take over the workload. For a stateful database workload, preparing a second VM alone is not sufficient. Two independent VMs cannot safely modify the same conventional disk image concurrently, and QEMU normally prevents simultaneous read-write access to an image to avoid storage corruption [QEMb]. The destination, therefore, requires its own writable storage. Changes made to the database at the source must then be transferred through a separate synchronization mechanism while the source remains active. In the PostgreSQL-based scenario considered in this thesis, this synchronization is provided through streaming replication: the primary records changes in its write-ahead log (WAL), while a standby receives and replays these changes against its own database state [Posb].

Preparing the destination shifts part of the transition cost to the period before switchover. The additional instance and the synchronization process consume resources while the source is still serving the workload, but can reduce the amount of initialization and state transfer required during the final transition. A prepared destination, therefore, trades additional preparation overhead and implementation complexity for the possibility of a shorter switchover without transferring the complete runtime state of the source VM.

2.5 PostgreSQL Memory and I/O Characteristics

PostgreSQL maintains database pages in a shared buffer pool that is accessible to its server processes. When a requested page is already present in these shared buffers, PostgreSQL can access it without first reading it into the buffer pool. Otherwise, the page is read through the operating system, where the corresponding database file contents may still be available in the guest operating system's page cache. The effectiveness of both caching layers depends on the active working set and the locality of database accesses [Posd].

Updates modify database pages in memory and generate write-ahead log (WAL) records describing those changes. WAL records must be made durable before the

corresponding modified data pages, allowing dirty buffer pages to be written back asynchronously rather than during every transaction commit. Dirty pages may be written by backend processes, the background writer, or during a checkpoint. Checkpoints ensure that data files reflect the WAL up to a defined point, limiting the amount of WAL that must be processed during crash recovery. These activities can create varying levels of memory and storage activity even under a constant request stream and may compete with foreground operations for system resources [Posh; Posd; Posg].

The buffer hit rate used in this thesis is the proportion of PostgreSQL buffer accesses satisfied by the shared buffer pool. It is calculated over each sampling interval as

$$\frac{\Delta\texttt{blks_hit}}{\Delta\texttt{blks_hit} + \Delta\texttt{blks_read}}.$$

The `blks_hit` counter records accesses for which the requested block was already present in PostgreSQL's buffer cache, while `blks_read` records blocks that had to be read into it. The metric does not include hits in the operating system's file system cache and therefore should not be interpreted as a direct physical-storage hit ratio [Posf].

2.6 Security and Failure Considerations

Live migration temporarily distributes VM state across the source, destination, and the transfer path, while prepared destinations introduce multiple copies of application state and additional coordination requirements. These mechanisms therefore differ in their exposure to component failures, unauthorized access, and inconsistent states. This section introduces the failure and security properties required for the later analysis.

2.6.1 Failure Dependencies

Migration strategies differ in which components must remain available for the VM to continue executing at a given point in the transition. Before a migration begins, the source contains the complete runtime state and represents the only active instance. During and after migration, parts of this state may be distributed between the source and destination, changing the windows and the consequences of process failure.

During most of a precopy migration, the source VM continues executing and retains the authoritative runtime state. A failure of the destination or the migration connection can therefore generally abort the transfer without losing the running workload. The critical transition occurs during the final stop-and-copy phase, when source execution is suspended, and the remaining state is transferred before execution resumes at the destination [Cla+05]. The stop-copy baseline, or cold migration, offloads all of this to the final downtime phase.

Postcopy reverses this relationship after the switchover, introducing a new risk of total migration failure. As the destination begins executing before the complete memory state has arrived, a failure of the source VM or migration process can make the migrated execution state unrecoverable, causing loss of availability and potentially loss of non-durable application progress. Its effect on persistent database consistency depends on the storage and recovery configuration. During postcopy, neither system alone contains all state required to continue independently. This means that a failure of the destination VM would also cause all progress made on this machine to be lost, even if the original source environment were resumed. This co-dependency persists until migration completes [HG09; QEMe].

2.6.2 Migration State and Trust Boundaries

The runtime state transferred during migration can contain substantially more than the application data stored on disk. Guest memory may include credentials, cryptographic material, database contents, recently processed requests, and other information that exists only temporarily during execution. Live migration, therefore, extends the trusted environment beyond the source host. The destination host, as well as the mechanism used to transport the migration state, becomes part of the security boundary. Depending on the deployment, this transport may range from a local Unix domain socket to a network connection protected by transport encryption and authentication [SM15; DGS23]. A party able to observe the migration state may gain access to sensitive information contained in guest memory, while unauthorized modification of the transferred state can affect the integrity or availability of the migrated VM. The confidentiality and integrity requirements of the migration path, therefore, depend on the trust relationship between source, destination, and the network between them.

The migration control path is similarly security-relevant. In the QEMU/KVM environment considered in this thesis, QMP provides machine-level control over a running QEMU instance and exposes operations that can alter VM execution and state. Access to this interface should accordingly be restricted in the same manner as other privileged VM or system management interfaces [QEMi]. The concrete transport and access controls used in the experimental environment are described in Chapter 4.

2.6.3 Postcopy Migration and `Userfaultfd`

As described in Section 2.3.3, postcopy migration requires the destination to access memory pages that may not yet be locally available. When the guest accesses such a page, execution must wait until the missing content has been retrieved from the source. On Linux, QEMU can use `userfaultfd` to participate in this process. This

interface allows a userspace process to register memory regions and receive notifications for selected page-fault events within them. Userspace can then resolve the fault by supplying the required page or otherwise making it available [Lind].

Page faults are ordinarily resolved by the kernel, whereas `userfaultfd` delegates the handling of some regions to userspace. This control is security-relevant. The Linux kernel documentation notes that allowing `userfaultfd` to intercept page faults triggered by kernel accesses to registered user-space memory has historically facilitated the exploitation of kernel vulnerabilities. Linux therefore provides several mechanisms for restricting access to the interface [Lind]. A secure postcopy deployment should consequently minimize the number of processes that can obtain the required `userfaultfd` capabilities. Granting broad access to unprivileged processes increases exposure, while a restricted configuration can limit access to the QEMU process responsible for migration. The configuration used in the experiments is described in Chapter 4.3.1.

2.6.4 Consistency and Split-Brain Risks

The prepared-destination approach introduces a different class of failure. As source and destination maintain separate writable storage and synchronize application state through replication, the transition must clearly determine which instance is authoritative. Before the destination is promoted, the source must stop accepting changes or otherwise be prevented from continuing as an independent primary. If both instances accept conflicting updates, their database states can diverge, creating a split-brain condition. As this threatens data consistency, practical systems commonly require an additional mechanism to prevent or isolate the old primary before the new instance becomes authoritative [Posa].

3 Related Work

Building on the concepts introduced in Chapter 2, this chapter reviews research on virtual-machine live migration, database-level relocation, and the security and failure properties of runtime-state transfer. It then positions this thesis relative to these existing approaches.

3.1 Virtual-Machine Live Migration

Clark et al. presented one of the foundational implementations of live virtual-machine migration using iterative precopy [Cla+05]. Their evaluation distinguishes total migration time, downtime, and interference with the running workload. It also identifies the guest's writable working set and available transfer bandwidth as important factors for migration convergence.

Hines et al. introduced postcopy migration, which transfers execution to the destination before all memory pages have arrived [HG09]. This avoids repeatedly transferring pages modified during migration but causes destination execution to depend on retrieving missing pages. Akoush et al. modeled migration duration and application impact based on workload and network characteristics [Ako+10]. Svärd et al. later compared precopy, postcopy, and hybrid approaches under different memory-update patterns [Svä+15].

Application-visible effects were examined by Voorsluys et al. using a multi-tier web application [Voo+09]. Their results show that live migration can affect application performance even when complete service unavailability remains brief.

3.2 Database-Level Relocation

Database relocation can also be performed within the database system, rather than transferring the complete runtime state of its virtual machine. Albatross considers multitenant databases in which the persistent database image resides on shared network storage. It transfers database cache contents and the state of active transactions to reduce service interruption and post-migration performance degradation [Das+11]. Zephyr instead addresses shared-nothing transactional databases, where persistent

data must also be moved, and combines on-demand data retrieval with asynchronous transfer of the remaining database state [Elm+11]. During migration, transaction processing is coordinated between the source and destination.

Madeus is a middleware approach to live database migration that operates under snapshot isolation [MF15]. Its evaluation with PostgreSQL examines migration under workloads that frequently update the data being transferred. Remus later proposed a live shard-migration protocol for distributed databases using snapshot isolation [Kan+22].

These approaches exploit information about transactions, database pages, and consistency requirements that is unavailable to a hypervisor. They generally introduce specialized migration protocols, whereas the prepared-destination mechanism evaluated in this thesis uses PostgreSQL streaming replication as an existing application-level relocation option.

3.3 Security and Failure Considerations

Live migration transfers volatile machine state between hosts, including guest memory, processor state, and device state. Research on secure live migration has therefore considered the confidentiality and integrity of the migration channel as well as the trustworthiness of the participating hosts. Sulaiman and Masuda propose protecting migration traffic using IPsec [SM15]. Deylami et al. present Kororā, a framework based on the Xen hypervisor that combines a virtual trusted platform module with integrity-monitoring components to protect the migration process [DGS23].

The migration mechanism also influences the consequences of failures. Precopy normally retains an executable source VM until the final switchover. In postcopy migration, execution moves to the destination before all memory pages have arrived, leaving the destination dependent on the source and migration connection until the remaining state has been transferred [HG09]. A failure during this interval can therefore prevent the VM from continuing on either host.

Failure handling is also part of database-level migration protocols such as Albatross and Zephyr, which coordinate source and destination state to preserve transactional correctness if relocation does not complete successfully [Das+11; Elm+11].

3.4 Positioning of this thesis

Taken together, the studies reviewed above primarily investigate how individual migration mechanisms can be implemented, optimized, evaluated, or secured. They pay less attention to the preceding provider-level decision about whether to preserve the complete VM state or to reconstruct the service using restart-based or database-specific mechanisms. This thesis examines that decision for an update-intensive PostgreSQL workload.

4 Methodology

This chapter outlines the experimental design used to compare virtual-machine live migration with restart-based and application-specific relocation strategies for a PostgreSQL workload. It describes the experimental environment, the implementation of the relocation mechanisms, the execution procedure, the measurement methods, and the evaluation criteria.

4.1 Research Design

This study used a controlled experimental comparison in which the relocation strategy was the principal experimental variable. Unless varied deliberately within a scenario, the VM, database, dataset, and workload configuration remained constant. Selected scenarios additionally varied workload intensity or migration parameters, such as the use of auto-converge and the delay before switching to postcopy. Each scenario was repeated ten times from the same initial disk state. Preparation requirements, operational complexity, failure dependencies, and security implications were assessed qualitatively from the implemented mechanisms, their transferred state, and their trust dependencies. No adversarial security experiments were performed.

4.2 Experimental Environment

The experiments used a PostgreSQL database running inside a QEMU/KVM virtual machine. The common VM, database, and workload configuration is summarized in Tables 4.1 and 4.2. A new writable overlay using the QEMU Copy-On-Write version 2 (`qcow2`) file format was created for each run so that all repetitions started from the same base image. The source and destination were separate QEMU processes on the same physical host, while the migration state was transferred through a Unix domain socket.

YCSB 0.17.0 generated the database workload via its PostgreSQL binding, which uses Java Database Connectivity (JDBC) to access the database. Because YCSB ran inside the guest, the same workload process continued across live migration. Restart-based relocation instead required launching a new YCSB instance once PostgreSQL became available again. PostgreSQL was configured with `full_page_writes=off`, rather than

its default value of `on`. This omits full-page images that PostgreSQL would otherwise write to WAL after checkpoints, reducing WAL volume at the cost of weaker protection against partial page writes following a system failure [Posg]. Because the prepared-destination strategy transfers and replays WAL, this setting may also have reduced its measured synchronization overhead relative to a default PostgreSQL configuration.

Because source and destination ran on the same physical host and migration used a Unix domain socket, the setup does not reproduce cross-host network exposure or the security controls of a production cloud deployment.

Table 4.1: Host and VM configuration

Component	**Configuration**
Host CPU	AMD Ryzen 7 7700X, 8-Cores
Host memory	32 GiB DDR5
Host operating system	Arch Linux x86_64, Linux 7.1.3-zen1-1-zen Kernel
Host storage	NVMe SSD using ext4
Virtualization	QEMU 11.0.2-2 with KVM acceleration
VM CPU	4 vCPUs, host CPU model
VM memory	8 GiB
Guest operating system	Arch Linux x86_64, Linux 7.1.3-zen1-1-zen Kernel
Virtual storage	qcow2 base image with a per-run writable overlay
Source and destination	Separate QEMU processes on the same physical host
Migration transport	Unix domain socket

Table 4.2: Database and Workload configuration

Component	**Configuration**
Database System	PostgreSQL 18.4
PostgreSQL configuration	`full_page_writes = off`; JSON logging in UTC; scenario-specific use of `pg_prewarm` and streaming replication
Workload	YCSB 0.17.0 using JDBC binding YCSB-based update-only workload
Dataset	2.5 million records
Database size	2925 MB
Worker threads	4 or 16, depending on scenario

4.3 Relocation Strategies

The evaluated scenarios implemented the relocation mechanisms introduced in Chapter 2 within a common experimental framework. The following sections describe the implementation differences between the relocation strategies that are relevant to interpreting the measurements. Table 4.3 gives an overview of the relevant QMP commands used to manage the virtual machines during the different approaches.

Table 4.3: Relevant QMP commands used to manage the guest systems

Command	Usage
`migrate`	Initiates migration to the destination
`query-migrate`	Polls migration status and statistics
`migrate-start-postcopy`	Switches an active migration to postcopy
`stop` / `cont`	Implements the explicit stop-copy baseline
`system_powerdown`	Requests clean shutdown
`migrate-set-capabilities`	Enables scenario-specific features

4.3.1 VM Migration Strategies

For all VM migration scenarios, the destination QEMU process was started in `incoming` migration mode. The destination waited for an incoming migration on the configured Unix domain socket. After configuring the required migration capabilities, the controller initiated migration through a QMP `migrate` command. The controller then queried the migration state periodically until QEMU reported completion. The virtual disk remained accessible to both QEMU processes and was not included in the migration stream.

Postcopy migration was enabled on both source and destination by sending `migrate-set-capabilities` through QMP and activating `postcopy-ram` as well as `postcopy-blocktime`. The first capability enables postcopy RAM migration, which depends on `userfaultfd`, while the second capability provides destination-side statistics on the time for which virtual CPUs were blocked by missing-page faults [QEMe; QEMg]. These capabilities were enabled on both QEMU instances before migration began in all postcopy scenarios. The controller first initiated the ordinary migration procedure and then requested a transition to postcopy once QEMU reported an active migration. In the immediate postcopy scenario, this request was issued at the first eligible polling cycle. The delayed variants instead allowed precopy to run for 5, 25, or 40 seconds before switching to postcopy. Destination execution resumed once QEMU

entered the `postcopy-active` state, while migration continued until the remaining memory had been transferred.

For the experiments, access to `userfaultfd` was enabled through `sysctl vm.unprivileged_userfaultfd=1`. This setting permits unprivileged processes to use the `userfaultfd` system call without the restrictions applied by the default value of zero [Lina]. It simplified the benchmark setup but broadened access beyond the QEMU migration process and, therefore, does not represent a hardened production configuration. The setting was applied only during the test session. A production deployment should instead restrict the required access to the migration process and its dependencies. The security relevance of `userfaultfd` is discussed in Section 2.6.3.

The experiments retained QEMU's default migration-bandwidth parameters. As a result, the precopy phase was limited to 128 MiB/s through `max-bandwidth`, while `max-postcopy-bandwidth` remained zero, indicating that no explicit limit was applied to the postcopy background transfer [QEMd; QEMg]. Since postcopy migration begins with a precopy phase, the precopy bandwidth limit remained active until the transition to postcopy.

A further precopy variant enabled QEMU's `auto-converge` capability. This allows QEMU to throttle source-vCPU execution when migration is not making sufficient progress, thereby reducing the rate at which the guest modifies memory. The experiment retained QEMU's default parameters: throttling began at 20%, increased in steps of 10 percentage points, and could reach a maximum of 99%. Throttling was triggered when the ratio of memory dirtied to memory transferred during the observation period reached 50%. The `cpu-throttle-tailslow` option remained disabled, so QEMU did not reduce the throttle increments during the final stage of convergence [QEMg].

4.3.2 Restart-Based Strategies

In the plain restart scenario, the source VM was shut down cleanly through QMP before a replacement VM was started from the same run-specific disk overlay. The controller waited until the guest was reachable through SSH and PostgreSQL accepted connections before launching a new YCSB workload instance.

The prewarmed-restart strategy followed the same procedure but used PostgreSQL's automatic prewarming mechanism to preserve part of the database cache state across the restart. PostgreSQL was configured to load the `pg_prewarm` module through `shared_preload_libraries`. Immediately before shutdown, the controller invoked `autoprewarm_dump_now()` to record the blocks currently present in PostgreSQL's shared buffers in the `autoprewarm.blocks` file. During destination startup, PostgreSQL's automatic prewarm workers loaded the recorded blocks back into the shared-buffer pool before the destination workload was launched [Posc]. The number of recorded

and restored blocks was retained to verify that the prewarming procedure completed successfully.

4.3.3 Prepared PostgreSQL Destination

The prepared-destination scenario used a second VM with a PostgreSQL standby and its own writable disk image, configured for streaming replication. The destination started while the source continued serving the workload, and database changes were transferred through the source's WAL. The standby received and replayed these WAL records against its own database state [Posb].

The controller first called `pg_is_in_recovery()` to verify that the destination remained in recovery. It then queried `pg_stat_replication` on the primary and calculated the difference between its flushed WAL position and the standby's reported replay position using `pg_wal_lsn_diff(pg_current_wal_flush_lsn(), replay_lsn)`. The handover began once the reported lag was no greater than 16 MiB, resulting in the source VM being shut down cleanly to prevent both instances from operating as primaries. The controller waited until all WAL already received by the standby had been replayed by comparing `pg_last_wal_receive_lsn()` with `pg_last_wal_replay_lsn()`. It then promoted the standby using `pg_promote(wait => true)` and confirmed that recovery had ended using `NOT pg_is_in_recovery()` [Pose]. Once the destination accepted connections as the new primary, a new YCSB workload instance was launched. This strategy prepared the persistent database state before the transition but did not transfer the source VM's execution state or PostgreSQL buffer contents.

4.4 Experimental Procedure and Measurement

The experiments were automated using two Python controllers. The script `bench_migration_qmp.py` implemented the VM migration scenarios, while `bench_raw_qmp.py` implemented the restart-based and prepared destination scenarios. The shell script `bench_runner.sh` defined the scenario parameters and invoked the appropriate controller for each experimental run. Each run began by creating the required writable disk overlays and starting the source VM. The controller waited until both the guest operating system was reachable and PostgreSQL accepted connections via SSH before launching the YCSB workload and the monitoring processes. After a scenario-specific pre-transition interval, the selected relocation procedure was initiated. Measurements continued throughout the transition and until the workload was completed. The resulting logs and statistics were then stored for later processing, and any remaining VM processes were terminated.

The workload was executed before relocation so that the database and benchmark were active when the transition began. The exact trigger time, workload concurrency, and operation count depended on the scenario. PostgreSQL statistics were collected separately to observe buffer behavior, and host-side sampling recorded resource usage of the source and destination QEMU processes. PostgreSQL server logs were retained for diagnostic purposes, including checkpoint and startup events. Destination-side transparent huge-page (THP) coverage was approximated as the ratio of AnonHugePages to resident set size (RSS) reported in `/proc/<destination-QEMU-pid>/smaps_rollup`. Because the destination QEMU process's RSS also includes allocations other than guest memory, this ratio is an indirect estimate of the proportion of guest-memory backing represented by anonymous huge pages rather than an exact measurement.

Two availability-related metrics were distinguished. For precopy and postcopy migration, VM downtime denotes the suspension interval reported by QEMU. For stop-copy, restart, and prepared-destination scenarios, service interruption was derived from host-side controller and workload events covering the interval during which the workload could not make progress across the transition.

Table 4.4: Relevant experimental metrics

Metric	Definition
Throughput	Completed YCSB operations per second.
Buffer hit rate	Interval-based PostgreSQL shared-buffer hit ratio.
VM downtime	Interval during which guest execution is suspended during migration, as reported by QEMU.
Service interruption	Host-observed interval during which the workload cannot make progress across stop-copy, restart, or prepared-destination relocation.
Migration duration	Time from migration initiation until QEMU reports completion.
Transition duration	Mechanism-specific interval from initiating relocation until the destination can serve the workload.
Transferred memory	Cumulative RAM transferred during the precopy and postcopy phases, as reported by QEMU.
Memory dirty rate	Rate at which the running workload dirties guest RAM pages.
Preparation overhead	Time and resources required before initiating the final transition.
I/O pressure	Host-side disk writes and I/O pressure-stall information collected during the experiment.

Table 4.5: Experimental setup

Scenario	Threads	Operation mix (100%)	Event time	Strategy
Baseline	4/16	updates	-	-
Main comparison	4	updates	150 s	baseline, precopy, postcopy, stop-copy
Stress comparison	16	updates	100 s	precopy, postcopy
Hybrid	16	updates	100 s	0/5/25/40 s postcopy delay, precopy (auto-converge)
Alternatives	4	updates	150 s	restart, prewarmed, prepared

4.5 Data Processing

The collected measurements were processed using Python analysis scripts that combined the YCSB output, QMP migration statistics, PostgreSQL samples, host-resource measurements, and controller event logs for each experimental run. Scalar metrics, such as migration duration, transferred memory, throughput during selected workload phases, buffer hit ratio, and service interruption, were first derived separately for each run. The resulting run-level values and their descriptive statistics were written to consolidated CSV files and used as the basis for the tables in Chapter 5.

Each run was treated as a single experimental observation, and 10 runs were attempted per scenario. One prepared destination run was excluded entirely because the source workload had already ended by the time of handover. Consequently, prepared aggregates use nine valid runs. Within valid runs, missing measurements were omitted only for the affected metric. For each scenario and metric, the analysis calculated the arithmetic mean, sample standard deviation, median, first and third quartiles, and the minimum and maximum. Continuous measurements with comparatively limited run-to-run variation are generally reported as mean $\pm$ standard deviation. Skewed, zero-heavy, or outlier-sensitive measurements, such as downtime, postcopy page requests, and postcopy block time, are instead reported as median with interquartile range (IQR). Missing measurements were omitted only for the affected metric, so the number of available observations may differ between individual results.

For the time-series figures, YCSB measurements retained their benchmark-relative elapsed-time axis, while controller and host timestamps were shifted relative to the recorded benchmark start. Samples were grouped into one-second elapsed-time buckets without interpolation. The figures show the pointwise median and IQR across the runs contributing to each bucket. To avoid increasingly sparse aggregates near the end of the workload, trailing buckets that were not represented by all included runs were omitted when they were missing data from two or more runs. Vertical event markers indicate the arithmetic mean of the corresponding event times across runs, while the displayed downtime and migration-duration annotations report the median. The processed results and event alignment were checked against the underlying logs for representative runs before inclusion in the evaluation.

The benchmark implementation, analysis and plotting scripts, experimental data, and setup instructions are available in the accompanying GitHub repository.

5 Evaluation

This chapter presents the results of the experimental comparison. It first characterizes the workload without relocation, since its phase changes and recurring disturbances provide the reference for interpreting later measurements. The subsequent sections compare migration mechanisms under moderate and stress conditions, examine convergence-control strategies, and evaluate restart-based alternatives. Section 4.5 describes the aggregation and interval-selection procedures.

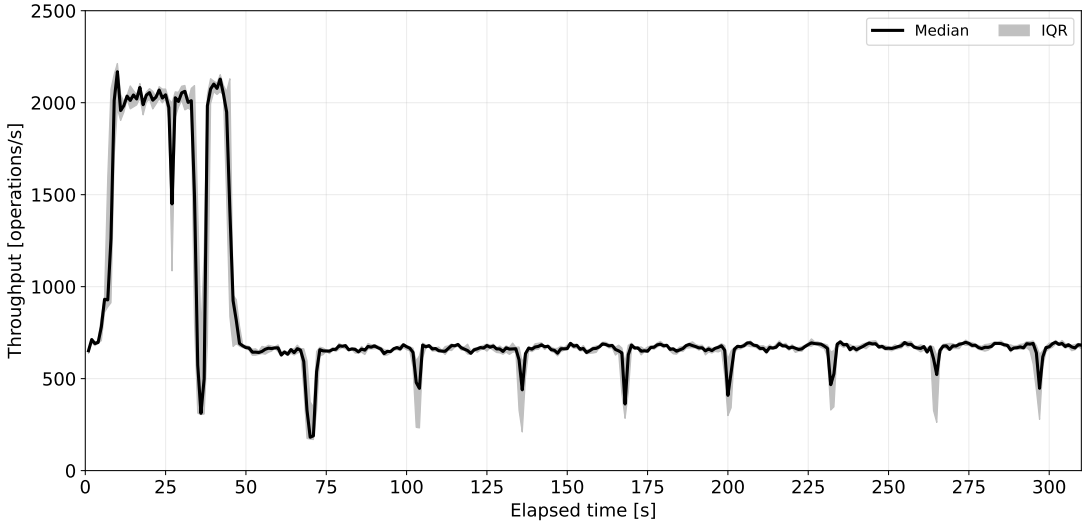
5.1 Baseline

The four-thread and sixteen-thread baselines exhibit the same overall structure in Figure 5.1: an initial high-throughput phase is followed by a substantially lower, comparatively stable operating phase. Since this transition occurs without relocation, it represents a workload-specific characteristic rather than migration-induced interference. Relocation events in the later experiments were therefore initiated only after the initial phase had ended.

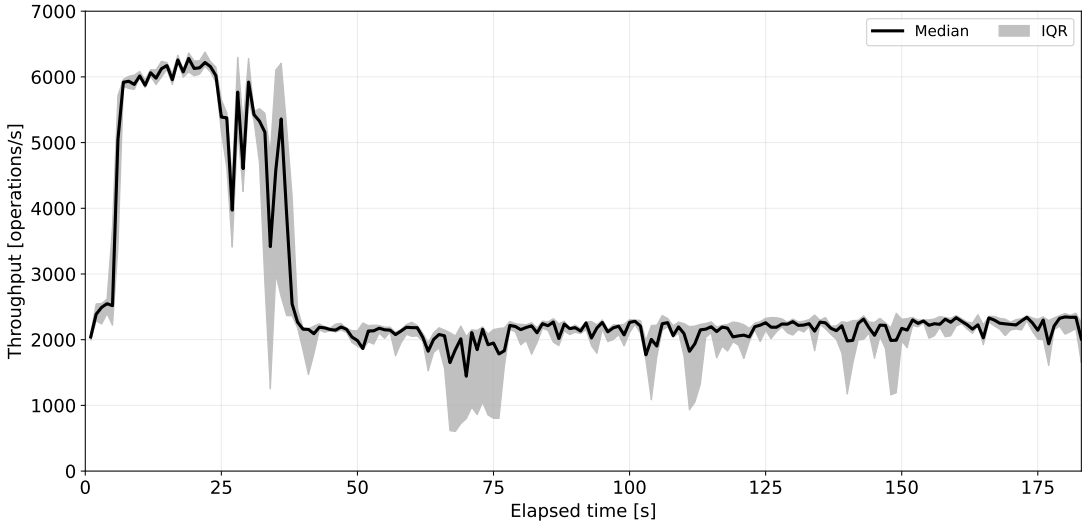
During the steady-state phase, the four-thread workload achieved 650.4 ± 8.6 operations/s and took 314.8 ± 7.3 s, while the sixteen-thread workload achieved 2089.4 ± 22.6 operations/s and took 189.1 ± 5.7 s. Nevertheless, both exhibit recurring throughput reductions, with timing varying between repetitions. The corresponding interquartile ranges widen where these disturbances are not aligned across runs.

The disturbances coincide with increased latency and host-side storage activity (see the I/O-pressure spikes before the dotted blue line in Figure 5.6b), but the available measurements do not identify PostgreSQL checkpointing, WAL flushing, guest write-back, or QEMU block-layer activity as their sole cause. They are therefore treated as workload-intrinsic, I/O-associated disturbances that may overlap with a migration or recovery interval without being caused by relocation. Across the relocation scenarios, latency showed no consistent additional pattern beyond the disturbances already reflected in throughput and is therefore not presented separately.

The two baselines are not necessarily directly comparable, as the regular workload was limited to 250,000 total operations and the stress workload to 500,000 to facilitate a reasonably long runtime. Note that due to these factors, the axis scales differ between the two baselines.



(a) Baseline workload with four YCSB worker threads.



(b) Baseline workload with sixteen YCSB worker threads.

Figure 5.1: Throughput of the baseline workloads.

5.2 Migration under Moderate Load

The main comparison evaluates precopy, immediate postcopy, and the stop-copy baseline using the four-thread workload. Figure 5.2 shows the throughput of each strategy together with the matching baseline, while Table 5.1 summarizes the migration and service-level metrics.

Table 5.1: Migration results under moderate load.
Values are mean ± standard deviation unless reported as median [IQR].

Strategy	Precopy	Postcopy	Stop-copy
Migration duration [s]	17.19 ± 0.23	1.869 ± 0.008	14.06 ± 0.09
Downtime [ms]	13.5 [12.0, 14.0]	4.0 [3.3, 5.0]	–
Service interruption [s]	–	–	14.49 ± 0.11
Total transferred [GiB]	2.19 ± 0.03	1.79 ± 0.01	1.78 ± 0.01
Precopy transferred [GiB]	2.16 ± 0.03	0.04 ± 0.00	–
Postcopy transferred [GiB]	–	1.75 ± 0.01	–
Page requests	–	2875 [2078, 3013]	–
Postcopy blocktime [ms]	–	11.5 [6.0, 20.5]	–
Throughput, during migration [ops/s]	622 ± 22	554 ± 19	–
Throughput, 30—90 s [ops/s]	628.4 ± 2.4	615.7 ± 6.0	(–)

5.2.1 Precopy Migration

Precopy completed successfully under the moderate workload and caused little foreground disruption (see Figure 5.2a). Throughput averaged 617.6 ± 8.6 operations/s before migration and 622 ± 22 operations/s while migration was active. Migration completed in 17.19 ± 0.23 s, with a median final interruption of only 13.5 ms.

QEMU transferred 2.19 ± 0.03 GiB, almost all of it during precopy. This volume was approximately 23% higher than for the stop-copy baseline, reflecting retransmission of pages modified during the iterative rounds, but it remained well below the VM's configured 8 GiB memory size. Late post-migration throughput returned to the same operating range as before migration. Under this workload, precopy therefore traded a moderate amount of transfer amplification for a short VM downtime and negligible sustained performance loss.

5.2.2 Immediate Postcopy Migration

Immediate postcopy moved execution to the destination after only 0.04 GiB had been transferred in precopy, while the remaining 1.75 ± 0.01 GiB arrived after the switchover.

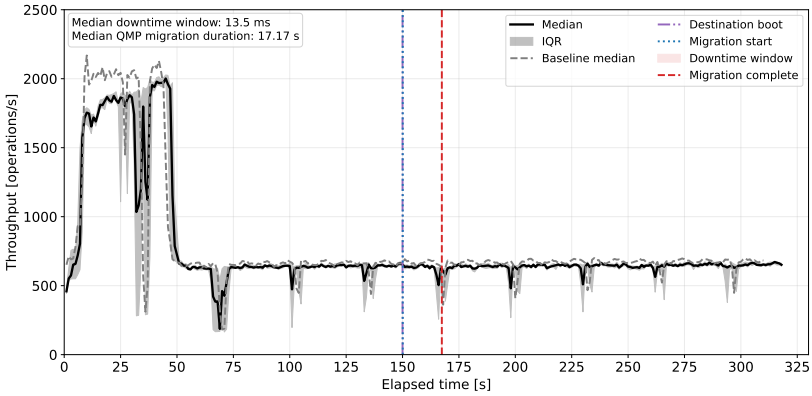
The cost instead appeared immediately after the switchover. The destination generated a median of 2875 postcopy page requests and accumulated a median of 11.5 ms of reported vCPU block time. Throughput while migration remained active was 554 ± 19 operations/s, approximately 9% below the scenario's pre-migration level. It recovered after the transfer and reached 615.7 ± 6.0 operations/s during the late post-migration window, remaining slightly below the corresponding precopy result.

To investigate the lower post-migration throughput observed after immediate postcopy, the host-side memory backing of the destination QEMU process was examined as an additional diagnostic. At migration completion, anonymous huge pages accounted for approximately 2.9% of its RSS and remained below 4% for approximately 40 s. This indicates that only a small fraction of the destination process memory was backed by THPs well after the migration had completed. The relationship between precopy duration and destination-side THP coverage is examined further under the stress workload in Section 5.4.1. The changed backing structure is consistent with a possible contribution to residual address-translation overhead, but the measurements do not isolate it from other migration-related or workload-intrinsic effects.

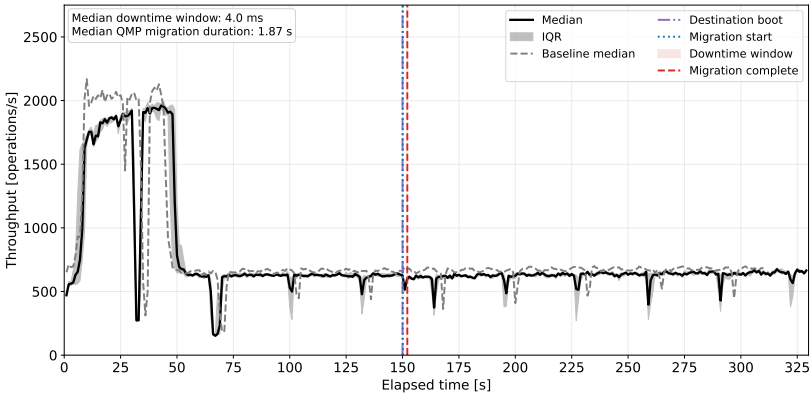
5.2.3 Stop-Copy Migration

The stop-copy baseline suspended the source before transferring the VM state. Because the guest could no longer modify memory, the mechanism avoided iterative retransmission and transferred 1.78 ± 0.01 GiB in 14.06 ± 0.09 s. Nearly the entire transfer occurred while the service was unavailable, and the host-measured interval from confirmed source suspension to destination resumption was 14.49 ± 0.11 s. As QEMU's internal downtime measurement excludes the manually introduced pause and is therefore not used as the service interruption for this strategy, the timing mentioned here is measured on the controller side as the interval from pausing the source to continuing the destination.

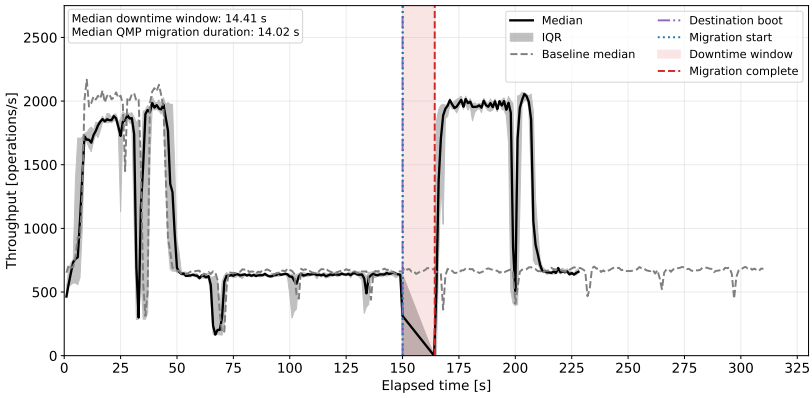
The post-resumption portion of Figure 5.2c requires cautious interpretation. Suspending the guest also suspended YCSB and its reporting thread, which distorts the benchmark's elapsed-time axis. However, cumulative operation counts and host-side timing indicate that the temporary acceleration after resumption was not only a reporting artifact. It is treated as an experiment-specific post-resumption transient rather than an inherent performance benefit of stop-copy. One possible explanation is that suspension and transfer altered guest- or host-side cache state, although the available measurements cannot distinguish this from other migration or reporting effects. The mechanism's robust conclusion is that it removes convergence uncertainty by directly exposing the memory-transfer interval as service unavailability.



(a) Precopy migration



(b) Postcopy migration



(c) Stop-Copy baseline migration
The benchmark-relative time axis is affected by guest suspension; see Section 5.2.3.

Figure 5.2: Throughput of the main migration strategies under moderate load

5.3 Migration under Stress

The stress experiment increased workload concurrency until the guest's dirtying activity approached the effective migration transfer capacity. Figure 5.3 compares precopy and postcopy under this condition, and Table 5.2 summarizes the migration statistics.

Table 5.2: Migration results under the sixteen-thread stress workload.
Values are mean ± standard deviation unless reported as median [IQR].

Strategy	Precopy	Postcopy
Duration [s]	109.8 ± 7.3	2.07 ± 0.04
Downtime [ms]	26.0 [23.5, 29.5]	4.5 [3.3, 6.8]
Total transferred memory [GiB]	13.85 ± 0.93	2.57 ± 0.06
Precopy transferred [GiB]	13.82 ± 0.93	0.04 ± 0.00
Postcopy transferred [GiB]	–	2.54 ± 0.06
Amplification [x guest RAM]	1.73 ± 0.12	0.322 ± 0.007
Median dirty rate [MiB/s]	127.15 ± 0.99	–
Transfer rate [MiB/s]	129.20 ± 0.15	1271 ± 22
Page requests	–	5036 [1981, 7193]
Postcopy blocktime [ms]	–	7.5 [5.3, 8.8]

5.3.1 Precopy Near the Convergence Limit

Under the stress workload, unrestricted precopy did not converge while the workload remained active (see Figure 5.3a). The median dirty-page rate was 127.1 MiB/s, while the effective transfer rate was approximately 129.0 MiB/s, leaving little margin for the remaining dirty set to decrease. In nine of ten runs, migration completed only within approximately two seconds of the detected benchmark end, while one run converged roughly 22 s earlier. The scenario is therefore described as near-non-convergent rather than as an unconditional migration failure. The measured dirty-page rate was close to the effective memory-transfer rate.

The limited convergence margin extended migration to 109.8 ± 7.3 s and increased the transferred volume to 13.85 ± 0.93 GiB, or approximately 1.73 times the configured guest memory of 8 GiB. Consequently, unrestricted precopy was completed primarily because workload activity ended, rather than because it established a stable convergence margin.

5.3.2 Postcopy under Stress

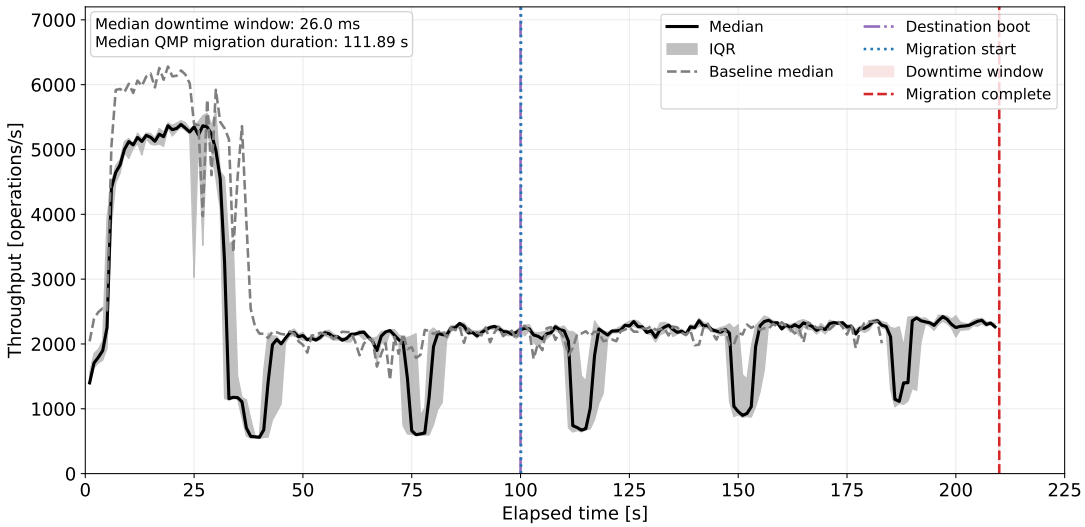
Postcopy completed while the stress workload remained active because progress no longer depended on reducing the source's dirty set. It reduced the mean migration duration from approximately 110 s to 2.07 ± 0.04 s and the total transfer from 13.85 GiB to 2.57 GiB. The destination nevertheless incurred demand-paging overhead, with a median of 5036 page requests and 7.5 ms of reported vCPU block time.

The resulting disturbance was concentrated around the switchover and the short postcopy interval. Late post-migration throughput recovered to 1979 ± 9 operations/s. As a result, postcopy bounded migration duration and transfer volume under a workload for which precopy had almost no convergence margin, but shifted part of the cost to destination-side execution and retained the failure dependency associated with incomplete postcopy state.

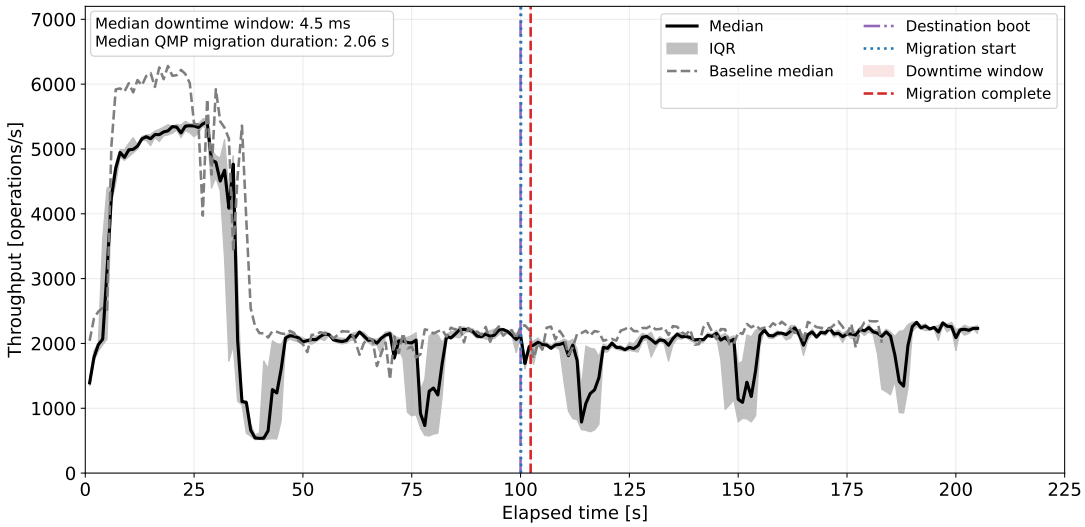
A clear difference was observed in the memory-transfer rates. As documented in Section 4.3.1, the precopy phase used QEMU's default bandwidth limit of 128 MiB/s, consistent with the measured rate of 129.20 ± 0.15 MiB/s. By contrast, `max-postcopy-bandwidth` was zero, so no explicit QEMU bandwidth limit applied after the switch to postcopy. Postcopy reached 1271 ± 22 MiB/s in the same-host setup, suggesting that its transfer rate was instead bounded by the effective capacity of the host memory subsystem and local transfer path.

One possible rationale for this asymmetry is that postcopy does not face the dirty-set convergence condition of precopy. Allowing the remaining memory transfer to proceed quickly also shortens the period of destination-side demand paging and source–destination co-dependency. This interpretation is not experimentally established as QEMU's design rationale.

As a diagnostic, the same sixteen-thread experiment was also repeated with a read-only workload. Precopy still completed only after the timed workload ended, with a dirtying rate of 129.5 ± 0.6 MiB/s. This does not imply that database reads modify the queried records; rather, the complete guest workload includes the Java client, network stack, operating system activity, and PostgreSQL execution state. The diagnostic does not isolate concurrency from dirtying behavior and is not treated as a separate mechanism comparison.



(a) Non-converging precopy under stress



(b) Postcopy under stress

Figure 5.3: Throughput of the main migration strategies under stress load

5.4 Convergence-Control Strategies

Two strategies were evaluated for workloads under which unrestricted precopy converged poorly: delaying the transition from precopy to postcopy and enabling QEMU's automatic convergence mechanism.

5.4.1 Delayed Postcopy

The delayed-postcopy variants allowed precopy to run for 5, 25, or 40 s before requesting the transition to postcopy. Figures 5.4a and 5.4b show representative timings, while Table 5.3 summarizes all three delays. Because the finite workload duration did not provide a complete 30—90 s post-migration window in every delayed-postcopy run, the corresponding throughput aggregates include only runs with complete measurements for that interval. The number of contributing runs, therefore, varies between variants, consistent with the metric-specific handling of missing measurements described in Section 4.5.

Table 5.3: Delayed-postcopy results under the sixteen-thread workload.
Values are mean ± standard deviation unless reported as median [IQR].

Postcopy delay	5 s	25 s	40 s
Migration duration [s]	6.30 ± 0.02	25.72 ± 0.09	40.494 ± 0.007
Total transferred [GiB]	2.75 ± 0.07	4.20 ± 0.12	5.20 ± 0.03
Precopy transferred [GiB]	0.64225 ± 0.00006	3.171 ± 0.006	5.0643 ± 0.0001
Postcopy transferred [GiB]	2.11 ± 0.07	1.03 ± 0.12	0.14 ± 0.03
Page requests [count]	979 [900, 1395]	0 [0, 0]	0 [0, 0]
Postcopy blocktime [ms]	3.0 [2.3, 3.0]	0 [0, 0]	0 [0, 0]
Throughput, first 10 s [ops/s]	1485 ± 247	2048 ± 75	1790 ± 210
Throughput, 30–90 s [ops/s]	2008 ± 12	2126 ± 45	2122 ± 23
THP coverage at +0 s [% of RSS]	5.9 [5.0, 7.6]	10.0 [9.5, 11.1]	24.4 [22.0, 25.3]
THP coverage at +40 s [% of RSS]	4.6 [4.0, 5.9]	23.2 [22.8, 24.0]	34.6 [33.3, 36.6]

Longer precopy phases progressively moved transfer work ahead of the switchover. Postcopy volume decreased from 2.11 ± 0.07 GiB after a 5 s delay to 1.03 ± 0.12 GiB after 25 s and 0.14 ± 0.03 GiB after 40 s. Correspondingly, demand-driven page requests fell from a median of 979 to zero in the 25 s and 40 s variants, although isolated requests in a small number of 25 s runs increased their arithmetic mean. The reduction in postcopy demand paging came at the cost of a longer overall migration and greater total traffic. Mean migration duration increased from 6.30 to 40.494 s, while total transfer rose from 2.75 to 5.20 GiB. Destination THP coverage 40 s after completion also

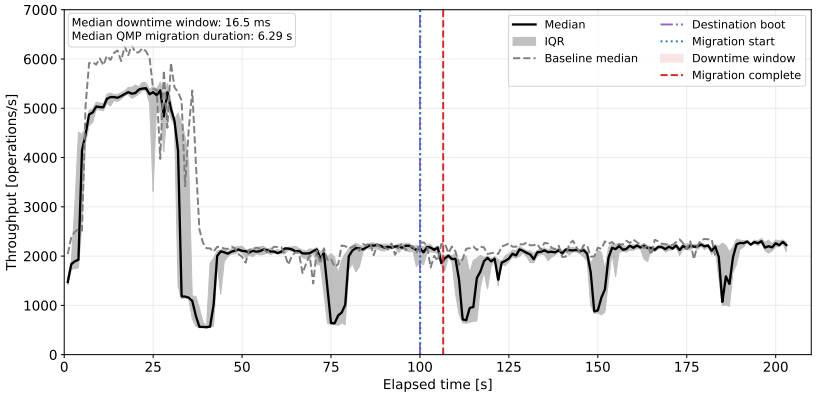
increased with the delay, from a median of 4.6% to 23.2% and 34.6%, respectively. This ordered relationship is consistent with less destination memory being discarded and repopulated at base-page granularity after a longer precopy phase.

Early post-migration throughput did not improve monotonically with the configured delay. The fixed evaluation windows overlapped differently with the recurring workload-intrinsic disturbances identified in Section 5.1, which prevents a clean ranking of the variants. Visual comparison of the plotted 5 s and 25 s variants nevertheless suggests a quicker return toward the pre-migration operating range when less state remained for postcopy. This pattern coincides with fewer demand-driven page requests and greater destination-side THP coverage, but the experiment does not isolate either as its cause. By 30–90 s after completion, all variants had returned to approximately 2000–2130 operations/s.

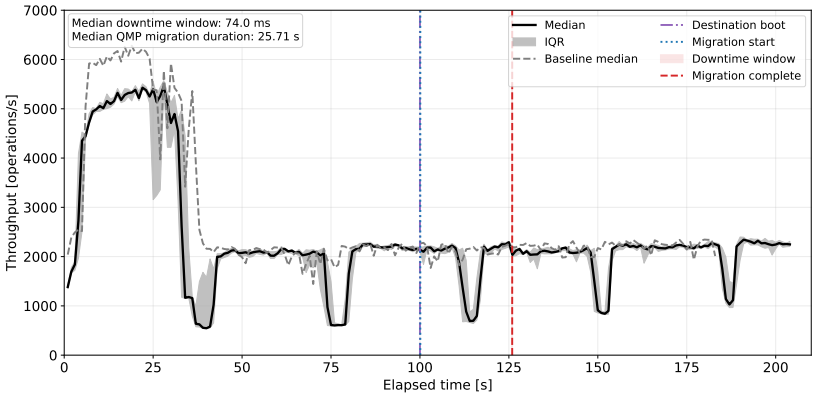
5.4.2 Automatic Convergence

QEMU's automatic convergence mechanism restored precopy progress by throttling the source vCPUs. The experiment retained QEMU's default throttling parameters, which are documented in the methodology. Figure 5.4c shows the workload throughput and throttling phase.

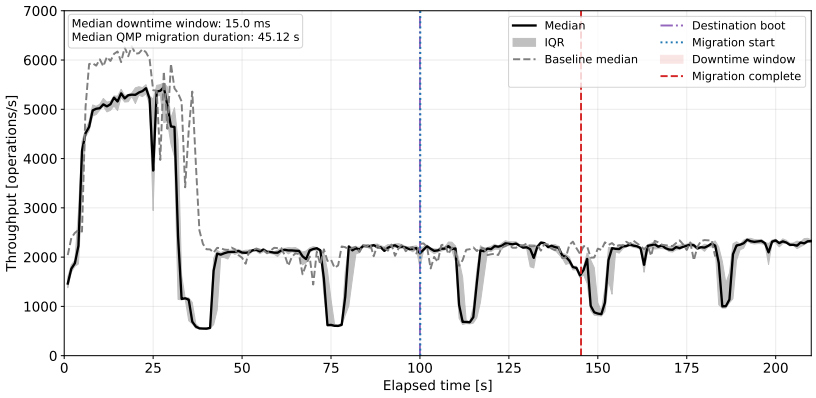
Throttling began 38.4 ± 1.2 s after migration started and remained active until completion for approximately 6.8 ± 0.8 s. The maximum observed throttle was $42.0 \pm 4.2\%$. The reported mean is not itself a configured throttle level, as QEMU's default auto-converge increment is set to 10%, resulting in maximum throttle values mostly at 40% and some at 50%. During the ten seconds preceding throttling, throughput averaged 2175 ± 18 operations/s; while throttling was active, it fell to 1892 ± 56 operations/s, a reduction of approximately 13%. This reduced vCPU execution rate lowered the median dirty-page rate to 63.7 ± 2.2 MiB/s while the migration stream remained near 129.75 ± 0.05 MiB/s. This restored a clear convergence margin and allowed migration to complete after 45.1 ± 1.1 s without switching to postcopy. Automatic convergence, therefore, retained precopy's source-authoritative execution model and avoided destination-side demand paging by deliberately reducing service capacity near the end of migration.



(a) Postcopy with a 5 s precopy deadline



(b) Postcopy with a 25 s precopy deadline



(c) Precopy with default auto-converge activated

Figure 5.4: Throughput of the hybrid migration strategies under stress load

5.5 Restart Alternatives

Restart-based strategies do not preserve complete VM execution state. Their costs instead arise from service initialization, cache reconstruction, or prior preparation of an application-level destination.

5.5.1 Cold and Prewarmed Restart

Figure 5.5 compares throughput and PostgreSQL buffer hit rate following a plain restart and a restart with cache prewarming. Table 5.4 summarizes the availability and recovery metrics.

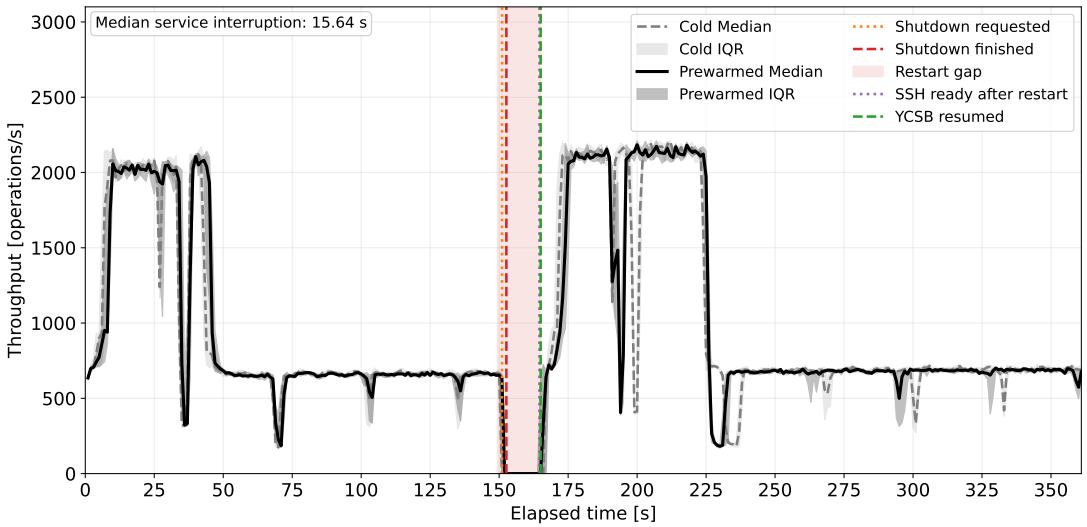
Table 5.4: Cold and prewarmed restart results. Values are mean ± standard deviation.

Strategy	Cold restart	Prewarmed restart
Service interruption [s]	15.16 ± 1.20	15.27 ± 1.16
First 10 s throughput [ops/s]	1305 ± 279	1011 ± 199
Later stabilized throughput [ops/s]	1265 ± 53	1330 ± 25
Buffer hit ratio, first 10 s [%]	85.37 ± 0.44	89.11 ± 0.24
Buffer hit ratio, 30–90 s [%]	86.68 ± 1.27	86.77 ± 1.20

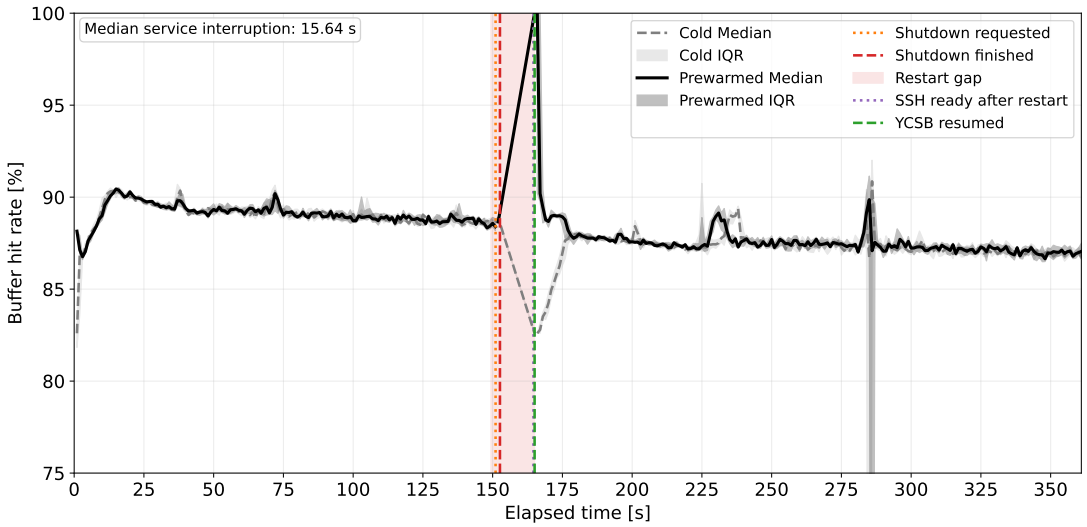
Prewarming did not shorten the interval needed to shut down the source, boot the replacement VM, and make PostgreSQL available. The automatic prewarming procedure completed successfully in all runs and restored nearly all recorded blocks before workload execution resumed.

Its clearer effect was on database cache state. During the first 10 s of destination execution, the pooled shared-buffer hit ratio was 85.37 ± 0.44 % after a cold restart and 89.11 ± 0.24 % after prewarming. This improvement did not translate into higher immediate throughput: the prewarmed scenario showed lower and more variable first-window throughput. During the later 30–90 s window, it achieved 1330 ± 25 operations/s compared with 1265 ± 53 operations/s after the cold restart.

Both restarted workloads re-entered YCSB's initial high-throughput phase, so these post-restart throughput values are not directly comparable with the pre-shutdown steady-state values. The buffer-hit measurements provide the cleaner evidence that prewarming reconstructed PostgreSQL shared-buffer contents, while the throughput results show that cache restoration alone did not determine short-term application performance.



(a) Throughput surrounding cold and prewarmed restart



(b) PostgreSQL shared buffer hit rate surrounding cold and prewarmed restart

Figure 5.5: Throughput and buffer hit-rate comparison of cold vs. prewarmed restart.

5.5.2 Prepared PostgreSQL Destination

The prepared-destination strategy moved VM startup and database synchronization ahead of the final handover. Figure 5.6 shows the resulting preparation, catch-up, shutdown, promotion, and recovery phases. Aggregates are based on the nine valid runs identified in Section 4.5.

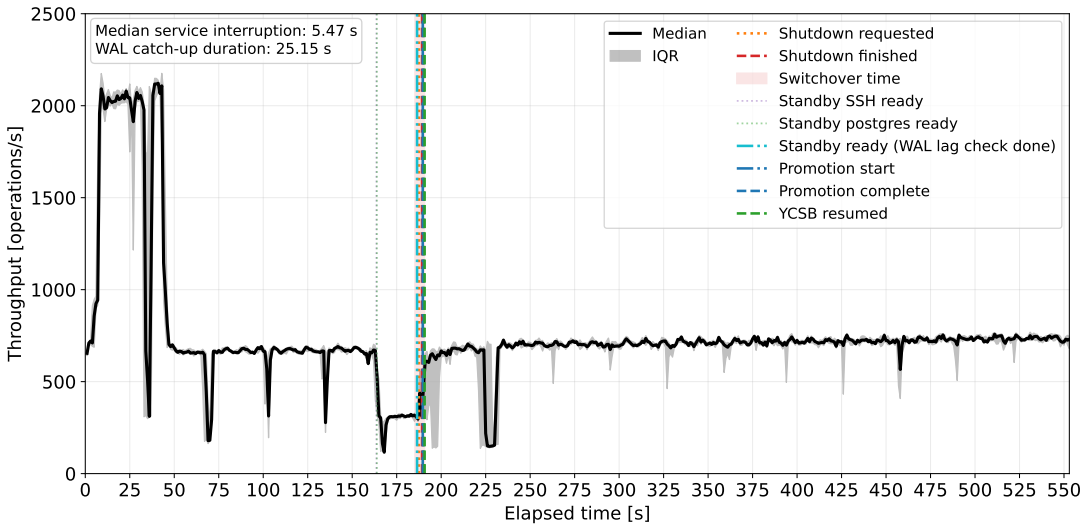
Before destination preparation, source throughput averaged 640.2 ± 5.4 operations/s. Merely booting the standby did not impose a measurable penalty: throughput during that phase was 653 ± 12 operations/s. The substantial cost began when PostgreSQL catch-up and WAL replay became active, reducing source throughput to 296.1 ± 8.6 operations/s. During the same phase, host writes increased from 19.45 ± 0.18 to 53.44 ± 2.59 MiB/s, and full I/O pressure increased from an average of $6.84 \pm 0.26\%$ to $10.94 \pm 0.68\%$. The measurements, therefore, associate the preparation penalty primarily with storage contention during synchronization rather than with starting a second VM alone.

Reaching the configured lag threshold required 36.5 ± 3.3 s from destination boot, after which the controlled handover itself took 3.23 ± 0.44 s. The observed service interruption was 5.52 ± 0.84 s, approximately 64% shorter than for the cold restart. Late destination throughput returned to 625 ± 45 operations/s, close to the source's pre-preparation range.

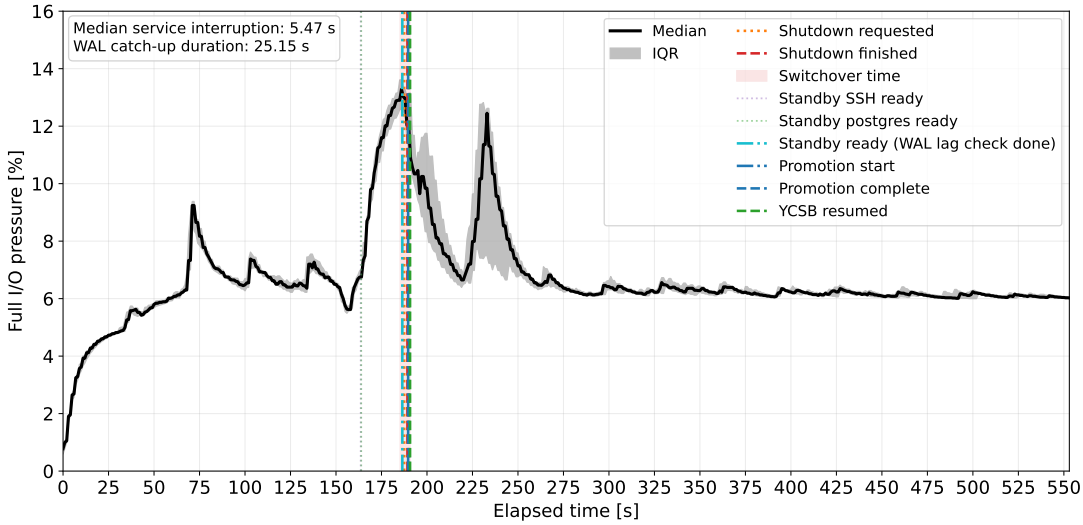
The destination was data-ready at promotion but did not inherit the source VM's execution state or PostgreSQL buffer contents. Its pooled hit ratio was $84.35 \pm 0.27\%$ during the first 10 s and recovered to $87.58 \pm 0.09\%$ during the late window. A write-intensive disturbance occurred after promotion in most runs and coincided with PostgreSQL checkpoint activity, although the measurements do not establish checkpointing as its sole cause.

Unlike the restart scenarios, the prepared destination did not reproduce the pronounced initial YCSB throughput spike. One possible explanation is that standby recovery and WAL replay had already accessed and partially populated parts of the database and operating-system cache hierarchy before the new workload began. If the initial spike reflects temporarily favorable conditions associated with a newly started workload and comparatively inactive or empty runtime state, suppressing this transient would ordinarily be desirable, even though it results in lower measured throughput during this benchmark-specific phase. The available measurements do not isolate this effect, however, and the destination's initial shared-buffer hit ratio remained below its later stabilized value.

The prepared strategy, therefore, shortened the final outage by performing work in advance, but shifted substantial I/O and throughput cost into the preparation interval and introduced the operational requirement to fence the old primary before promotion.



(a) YCSB throughput surrounding the prepared switchover



(b) Host full I/O pressure during standby catch-up and switchover

Figure 5.6: Throughput and I/O pressure of the prepared destination scenario

5.6 Evaluation Summary

Under moderate load, precopy completed with a low-millisecond interruption and little sustained throughput loss, whereas stop-copy exposed almost the complete transfer interval as service unavailability. Under stress, precopy approached its convergence boundary, substantially increasing migration duration and the amount of memory transferred. Postcopy completed quickly under the same workload, but caused short-lived destination-side demand paging. Delaying the transition to postcopy reduced page requests at the cost of greater migration duration and transfer volume, while automatic convergence restored precopy progress by throttling source execution.

The restart-based strategies produced longer service interruptions. Cache prewarming increased the initial PostgreSQL shared-buffer hit ratio but did not improve immediate throughput. Preparing a replicated PostgreSQL destination reduced the final service interruption relative to a restart, but its catch-up phase caused substantial storage contention and reduced source throughput.

6 Discussion and Conclusion

This chapter interprets the experimental findings with regard to state preservation, service interruption, resource overhead, as well as failure and security exposure. It then consolidates the resulting trade-offs and answers the research question from the perspective of a cloud database provider. The findings characterize the evaluated mechanisms and setup rather than predicting the performance of a complete distributed database service.

6.1 State Preservation versus Reconstruction

The state-preserving mechanisms differ primarily in how they ensure migration progress and when execution moves to the destination. Precopy retains the source as the authoritative execution environment until the final switchover, but its progress depends on a sufficient margin between the memory-transfer rate and the workload's dirty-page rate. Postcopy removes this convergence dependency by transferring execution earlier, but introduces destination-side demand paging and stronger operational and failure dependencies. Its required capabilities, including access to `userfaultfd`, must also be configured before migration begins. QEMU's auto-converge capability and guest vCPU dirty-rate limiting can force precopy convergence by reducing the rate at which the guest modifies memory. From the perspective of a serverless database provider, however, this is not merely an internal migration cost: throttling reduces the compute capacity available to the customer and may therefore appear as lower database throughput or increased request latency during migration. The provider must consequently choose between making migration progress customer-visible through temporary workload restriction, accepting postcopy's destination-side performance costs and additional failure and security dependencies, or allocating greater migration bandwidth at the risk of increased host or network contention.

Among the reconstruction-based mechanisms, a prepared destination offered the strongest reduction in service interruption. Its viability, however, depends on whether the preparation cost is acceptable under the running workload. In the evaluated setup, the source and destination disk images shared a single physical storage device, which likely amplified the observed I/O contention. Plain and prewarmed restarts remain

simpler alternatives when operational simplicity is more important than preserving execution and cache state.

6.2 Service Interruption

Precopy and postcopy both achieved low-millisecond migration downtime, whereas stop-copy and the restart-based procedures produced service interruptions measured in seconds. The prepared destination substantially reduced the restart-based service gap but did not approach the interruption times of successful live migrations. These values represent different portions of the respective procedures: live-migration downtime primarily measures VM suspension, whereas restart-based service gaps also include destination initialization and workload resumption.

Postcopy achieved the shortest and most consistent migration duration and downtime because execution resumed after only the minimum required state had been transferred. In the evaluated configuration, its remaining memory transfer was also not subject to the 128 MiB/s limit applied during precopy. This default asymmetry is consistent with the different progress characteristics of the mechanisms. A poorly converging precopy migration may retransmit modified pages for an extended period, so limiting its bandwidth bounds the sustained overhead it can impose. Postcopy does not face this limitation, and allowing the remaining transfer to proceed quickly shortens the period of destination-side page faults and source–destination co-dependency. This explanation is an interpretation of the default configuration.

6.3 Resource Overheads

A cold restart introduced the least additional relocation overhead because it required neither memory transfers nor continuous destination preparation. Buffer prewarming increased destination-side I/O slightly, but remained negligible on the host. Preparing a destination incurred the highest host overhead among the evaluated strategies, as it required two virtual machines to run simultaneously and to constantly transfer a database stream, which created high storage I/O pressure.

The migration mechanisms exposed different resource profiles. Precopy consumed bandwidth throughout the migration and, near its convergence boundary, retransmitted enough modified memory to exceed the configured guest memory size. Automatic convergence bounded this behavior by reducing source execution capacity. Postcopy avoided iterative retransmission, but concentrated transfer activity and demand-paging overhead after the switchover. Reduced destination-side THP coverage may also have contributed to residual overhead, although its independent effect could not be

isolated. A provider must therefore consider not only total resource consumption, but also whether that consumption appears as sustained pre-switchover load, a short post-switchover burst, or deliberate workload throttling.

6.4 Failure and Security Exposure

The following comparison concerns mechanism-specific security and failure exposure rather than the effectiveness of an end-to-end production security deployment. A non-trivial difference among the tested mechanisms lies in how they extend the failure domain. Precopy retains the source as the authoritative instance until the final switchover and can generally be aborted while the VM continues running at the source. Postcopy resumes execution before all memory is available at the destination, temporarily making the VM dependent on both the source host and the migration channel. A failure during this phase can therefore render the VM unrecoverable. Live migration also transfers the complete guest memory, which may contain credentials, cryptographic material, database contents, and other transient information. It requires that both the migration and control channels be protected against disclosure and manipulation.

In the evaluated QEMU implementation, postcopy additionally depended on the `userfaultfd` system call for userspace-assisted handling of missing page faults at the destination. This introduces another security-relevant capability that should be granted only to the processes requiring it. The unprivileged access enabled for the experiments simplified the benchmark configuration but should not be interpreted as a hardened production setup.

Restart-based approaches avoid these migration-specific dependencies but rely on durable storage and correct recovery after a longer interruption. A prepared destination introduces a different coordination risk: the old primary must be prevented from continuing independently before the standby is promoted. Its principal failure concerns are therefore replication lag, promotion correctness, and protection against split-brain operation.

6.5 Final Comparison

Table 6.1 consolidates the experimental results and the qualitative implications discussed in the preceding sections. Service interruption, active-service interference, host-resource overhead, and outcome uncertainty are informed primarily by the measurements. Preparation, post-transition reconstruction, failure exposure, security exposure, and operational complexity additionally reflect the implementation requirements and mechanism-specific dependencies identified in Chapters 2 and 4.

The ratings are relative to the evaluated setting and should be interpreted within each row. The dimensions are not sufficiently commensurable to derive an aggregate score or universal ranking.

Table 6.1: Qualitative comparison of the evaluated basic relocation mechanisms

	Precopy migration	Postcopy migration	Stop-copy migration	Cold restart	Prewarmed restart	Prepared switchover
Preparation	Low	Medium	Low	Low	Medium	High
Active-service interference[1]	Low	Medium	Low	Low	Low	High
Post-transition reconstruction[2]	Low	Low	Low	High	High[3]	Medium
Host overhead	High	Medium	Medium	Low	Low	High
Service interruption	Low	Low	High	High	High	Medium
Outcome uncertainty[4]	High	Medium	Low	Medium	Medium	Medium
Failure exposure	Medium	High	Medium	Low	Low	Medium
Security exposure	Medium	High	Medium	Low	Low	Medium
Operational complexity	Medium	High	Medium	Low	Medium	High

[1] Excludes periods of complete unavailability, which are assessed separately as service interruption.
[2] Refers to performance- and execution-relevant runtime state that must be rebuilt after handover, not to persistent database state.
[3] Prewarming increased the initial PostgreSQL shared-buffer hit ratio but did not improve throughput or service interruption in the evaluated setup. Its effectiveness may differ for larger working sets or alternative implementations.
[4] Includes measured run-to-run variation and mechanism-specific uncertainty, such as workload-dependent precopy convergence.
Green indicates a comparatively favorable characteristic, yellow an intermediate or conditional characteristic, and red a comparatively unfavorable characteristic. The colors support comparison within each row and do not imply an overall ranking between mechanisms.

6.6 Answer to the Research Question

This thesis investigated how QEMU/KVM live migration compares with restart-based and database-level relocation mechanisms for an update-intensive PostgreSQL workload, and under which operational conditions preserving complete VM runtime state is justified. The results support a conditional strategy rather than a universally preferable mechanism. Precopy represents the most suitable initial state-preserving approach

when sufficient convergence margin exists because it combines short interruption with a source-authoritative execution model. If the workload's dirty-page rate approaches the available transfer capacity, the provider must either accept a temporary customer-visible reduction in service capacity to force precopy convergence or switch to postcopy. The latter bounds migration duration and transfer amplification, but does so at the cost of destination-side interference and stronger operational, failure, and security dependencies. Preserving complete VM state is therefore most strongly justified when a very short service interruption is a primary requirement, and the required transfer capacity, operational controls, and security safeguards are available.

Where these conditions do not hold, reconstruction-based mechanisms remain reasonable alternatives. A cold restart offers the lowest migration-specific complexity but causes a longer interruption. Cache prewarming restored part of PostgreSQL's buffer state without improving immediate throughput in the evaluated setup. A prepared PostgreSQL destination reduced restart-based interruptions but shifted substantial cost to destination preparation and storage activities. The appropriate mechanism is consequently determined by which cost is acceptable during deployment: service interruption, active-service interference, host resource consumption, operational complexity, or migration-specific exposure.

6.7 Limitations and Future Work

Source and destination were executed as separate QEMU processes on the same physical host, and migration used a Unix domain socket. The experiments, therefore, do not reproduce cross-host network latency, bandwidth contention, encrypted migration transport, or independent host failures. The study also evaluated one PostgreSQL configuration and an update-only YCSB workload, limiting the extent to which the measured values can be generalized to other workloads or database systems.

Future work should first repeat the prepared-destination experiment using separate storage devices of comparable performance to distinguish replication overhead from shared-device contention. Further experiments could investigate whether standby recovery, WAL replay, or changes in cache state explain the prepared-destination and stop-copy throughput transients. The possible contribution of destination-side THP coverage to postcopy performance should also be isolated by controlling or disabling THP. Further studies could evaluate cross-host migration under realistic network and encrypted-transport conditions, additional workload and database configurations, adaptive postcopy switching criteria, dirty-rate limiting, and explicit failure injection during postcopy migration and database promotion.

Bibliography

[Ako+10] S. Akoush, R. Sohan, A. Rice, A. W. Moore, and A. Hopper. "Predicting the Performance of Virtual Machine Migration." In: *2010 IEEE International Symposium on Modeling, Analysis and Simulation of Computer and Telecommunication Systems*. 2010, pp. 37–46. DOI: 10.1109/MASCOTS.2010.13.

[Ant+19] P. Antonopoulos, A. Budovski, C. Diaconu, A. Hernandez Saenz, J. Hu, H. Kodavalla, D. Kossmann, S. Lingam, U. F. Minhas, N. Prakash, V. Purohit, H. Qu, C. S. Ravella, K. Reisteter, S. Shrotri, D. Tang, and V. Wakade. "Socrates: The New SQL Server in the Cloud." In: *Proceedings of the 2019 International Conference on Management of Data*. SIGMOD '19. Amsterdam, Netherlands: Association for Computing Machinery, 2019, pp. 1743–1756. ISBN: 9781450356435. DOI: 10.1145/3299869.3314047.

[Cla+05] C. Clark, K. Fraser, S. Hand, J. G. Hansen, E. Jul, C. Limpach, I. Pratt, and A. Warfield. "Live migration of virtual machines." In: *Proceedings of the 2nd Conference on Symposium on Networked Systems Design & Implementation - Volume 2*. NSDI'05. USA: USENIX Association, 2005, pp. 273–286. DOI: 10.5555/1251203.1251223.

[Das+11] S. Das, S. Nishimura, D. Agrawal, and A. El Abbadi. "Albatross: lightweight elasticity in shared storage databases for the cloud using live data migration." In: *Proc. VLDB Endow.* 4.8 (May 2011), pp. 494–505. ISSN: 2150-8097. DOI: 10.14778/2002974.2002977.

[DGS23] H. Deylami, J. Gutierrez, and R. Sinha. "Kororā: A secure live virtual machine job migration framework for cloud systems integrity." In: *Array* 19 (2023), p. 100312. ISSN: 2590-0056. DOI: 10.1016/j.array.2023.100312.

[Elm+11] A. J. Elmore, S. Das, D. Agrawal, and A. El Abbadi. "Zephyr: live migration in shared nothing databases for elastic cloud platforms." In: *Proceedings of the 2011 ACM SIGMOD International Conference on Management of Data*. SIGMOD '11. Athens, Greece: Association for Computing Machinery, 2011, pp. 301–312. ISBN: 9781450306614. DOI: 10.1145/1989323.1989356.

[HG09] M. R. Hines and K. Gopalan. "Post-copy based live virtual machine migration using adaptive pre-paging and dynamic self-ballooning." In: *Proceedings of the 2009 ACM SIGPLAN/SIGOPS International Conference on Virtual Execution Environments*. VEE '09. Washington, DC, USA: Association for Computing Machinery, 2009, pp. 51–60. ISBN: 9781605583754. DOI: 10.1145/1508293.1508301.

[Kan+22] J. Kang, L. Cai, F. Li, X. Zhou, W. Cao, S. Cai, and D. Shao. "Remus: Efficient Live Migration for Distributed Databases with Snapshot Isolation." In: *Proceedings of the 2022 International Conference on Management of Data*. SIGMOD '22. Philadelphia, PA, USA: Association for Computing Machinery, 2022, pp. 2232–2245. ISBN: 9781450392495. DOI: 10.1145/3514221.3526047.

[Lina] Linux Kernel Developers. *Documentation for /proc/sys/vm/*. URL: https://docs.kernel.org/admin-guide/sysctl/vm.html (visited on 07/20/2026).

[Linb] Linux Kernel Developers. *The Definitive KVM (Kernel-based Virtual Machine) API Documentation*. URL: https://docs.kernel.org/virt/kvm/api.html (visited on 07/20/2026).

[Linc] Linux Kernel Developers. *Transparent Hugepage Support*. URL: https://docs.kernel.org/admin-guide/mm/transhuge.html (visited on 07/20/2026).

[Lind] Linux Kernel Developers. *Userfaultfd*. URL: https://docs.kernel.org/admin-guide/mm/userfaultfd.html (visited on 07/20/2026).

[MF15] T. Mishima and Y. Fujiwara. "Madeus: Database Live Migration Middleware under Heavy Workloads for Cloud Environment." In: *Proceedings of the 2015 ACM SIGMOD International Conference on Management of Data*. SIGMOD '15. Melbourne, Victoria, Australia: Association for Computing Machinery, 2015, pp. 315–329. ISBN: 9781450327589. DOI: 10.1145/2723372.2724578.

[Mic] Microsoft. *What Is Azure HorizonDB?* Preview documentation. URL: https://learn.microsoft.com/en-us/azure/horizondb/overview (visited on 07/20/2026).

[Neo] Neon. *Neon Architecture*. URL: https://neon.com/docs/introduction/architecture-overview (visited on 07/20/2026).

[Posa] PostgreSQL Global Development Group. *Failover*. URL: https://www.postgresql.org/docs/18/warm-standby-failover.html (visited on 07/20/2026).

[Posb] PostgreSQL Global Development Group. *Log-Shipping Standby Servers*. URL: https://www.postgresql.org/docs/18/warm-standby.html (visited on 07/20/2026).

[Posc] PostgreSQL Global Development Group. *pg_prewarm: Preload Relation Data into Buffer Caches*. URL: https://www.postgresql.org/docs/18/pgprewarm.html (visited on 07/20/2026).

[Posd] PostgreSQL Global Development Group. *Resource Consumption*. URL: https://www.postgresql.org/docs/18/runtime-config-resource.html (visited on 07/20/2026).

[Pose] PostgreSQL Global Development Group. *System Administration Functions*. URL: https://www.postgresql.org/docs/18/functions-admin.html (visited on 07/20/2026).

[Posf] PostgreSQL Global Development Group. *The Cumulative Statistics System*. URL: https://www.postgresql.org/docs/18/monitoring-stats.html (visited on 07/20/2026).

[Posg] PostgreSQL Global Development Group. *Write Ahead Log: Checkpoints*. URL: https://www.postgresql.org/docs/18/runtime-config-wal.html (visited on 07/20/2026).

[Posh] PostgreSQL Global Development Group. *Write-Ahead Logging (WAL)*. URL: https://www.postgresql.org/docs/18/wal-intro.html (visited on 07/20/2026).

[QEMa] QEMU Project. *Dirty Limit*. URL: https://qemu.readthedocs.io/en/master/devel/migration/dirty-limit.html (visited on 07/20/2026).

[QEMb] QEMU Project. *Disk Images*. URL: https://qemu.readthedocs.io/en/master/system/images.html (visited on 07/20/2026).

[QEMc] QEMU Project. *Migration Framework*. URL: https://qemu.readthedocs.io/en/master/devel/migration/main.html (visited on 07/20/2026).

[QEMd] QEMU Project. *Migration Option Defaults in QEMU 11.0.2*. Source file migration/options.c. URL: https://gitlab.com/qemu-project/qemu/-/blob/v11.0.2/migration/options.c (visited on 07/20/2026).

[QEMe] QEMU Project. *Postcopy*. URL: https://www.qemu.org/docs/master/devel/migration/postcopy.html (visited on 07/20/2026).

[QEMf] QEMU Project. *QEMU Machine Protocol Specification*. URL: https://qemu.readthedocs.io/en/master/interop/qmp-spec.html (visited on 07/20/2026).

[QEMg] QEMU Project. *QEMU QMP Reference Manual*. URL: https://qemu.readthedocs.io/en/master/interop/qemu-qmp-ref.html (visited on 07/20/2026).

[QEMh] QEMU Project. *QEMU System Emulation: Introduction*. URL: https://qemu.readthedocs.io/en/master/system/introduction.html (visited on 07/20/2026).

[QEMi] QEMU Project. *Security*. URL: https://www.qemu.org/docs/master/system/security.html (visited on 07/20/2026).

[SM15] N. A. b. Sulaiman and H. Masuda. "Evaluation of A Secure Live Migration of Virtual Machines Using IPsec Implementation." In: *International Journal of Networked and Distributed Computing* 3.2 (Apr. 2015), pp. 99–108. ISSN: 2211-7946. DOI: 10.2991/ijndc.2015.3.2.4.

[Svä+15] P. Svärd, B. Hudzia, S. Walsh, J. Tordsson, and E. Elmroth. "Principles and Performance Characteristics of Algorithms for Live VM Migration." In: *SIGOPS Oper. Syst. Rev.* 49.1 (Jan. 2015), pp. 142–155. ISSN: 0163-5980. DOI: 10.1145/2723872.2723894.

[Ver+17] A. Verbitski, A. Gupta, D. Saha, M. Brahmadesam, K. Gupta, R. Mittal, S. Krishnamurthy, S. Maurice, T. Kharatishvili, and X. Bao. "Amazon Aurora: Design Considerations for High Throughput Cloud-Native Relational Databases." In: *Proceedings of the 2017 ACM International Conference on Management of Data*. SIGMOD '17. Chicago, Illinois, USA: Association for Computing Machinery, 2017, pp. 1041–1052. ISBN: 9781450341974. DOI: 10.1145/3035918.3056101.

[Voo+09] W. Voorsluys, J. Broberg, S. Venugopal, and R. Buyya. "Cost of Virtual Machine Live Migration in Clouds: A Performance Evaluation." In: *Cloud Computing*. Ed. by M. G. Jaatun, G. Zhao, and C. Rong. Berlin, Heidelberg: Springer Berlin Heidelberg, 2009, pp. 254–265. ISBN: 978-3-642-10665-1. DOI: 0.1007/978-3-642-10665-1_23.

Declaration of Generative AI Use

Generative AI tools were used as supporting aids during the preparation of this thesis. Their use is documented in the table below. AI-generated and AI-assisted code, suggestions, interpretations, and textual revisions were critically reviewed and verified. The author remains fully responsible for the methodology, results, interpretations, and final text.

Generative AI tool	Details	
OpenAI ChatGPT	**Use case:**	Brainstorming of implementation approaches and possible interpretations; literature-search assistance; structural and linguistic revision
	Scope:	Selected implementation and interpretation discussions, as well as selected text passages
	Comments:	AI outputs were used as suggestions and editing assistance. All proposed interpretations, technical claims, structural changes, and textual revisions were critically reviewed and adapted by the author. The final argumentation and conclusions were determined by the author and checked against the experimental results and original literature, as well as authoritative documentation.
OpenAI Codex	**Use case:**	Development and revision of host-side logging using `psutil`, data-processing, analysis, and plotting scripts; provision of selected code snippets and shell commands
	Scope:	Host-side resource logger; selected analysis and plotting scripts; individual sections of the benchmark and experimental-control scripts
	Comments:	The benchmark design and the main experimental-control scripts were developed primarily by the author. Generative AI was used for selected code snippets and longer shell commands. The host-side logger and substantial parts of the analysis and plotting code were generated with AI assistance. All generated or modified code was reviewed and validated against the underlying logs and expected program behavior.

Version/model: Different models were used during the preparation of the thesis. Exact model versions were not consistently recorded.